

KOCAELİ ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

ARAŞTIRMA PROBLEMLERİ / BİTİRME TEZİ

YAPAY ZEKA İLE COVID-19 TESPİTİ

HATİCE AKILLI

KOCAELİ 2024

KOCAELİ ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

ARAŞTIRMA PROBLEMLERİ / BİTİRME TEZİ

YAPAY ZEKA İLE COVID-19 TESPİTİ

HATİCE AKILLI

ÖNSÖZ VE TEŞEKKÜR

Covid-19 pandemisi, dünya genelinde sağlık sistemleri ve toplumlar üzerinde derin bir etki bırakmıştır. Bu süreçte, hastalığın erken tespiti ve etkin yönetimi, salgının kontrol altına alınmasında kritik bir rol oynamıştır. Gelişen yapay zeka ve derin öğrenme teknolojileri, özellikle tıbbi görüntüleme alanında büyük bir potansiyele sahiptir. Bu proje, Covid-19'un tespitinde kullanılabilecek bir derin öğrenme modelinin geliştirilmesini ve uygulanmasını amaçlamaktadır. Bu çalışmada, bilgisayarlı görme tekniklerinden yararlanılarak akciğer radyografi görüntüleri üzerinde CNN yöntemleri ile sınıflandırma yapılmıştır.

Öncelikle, bu çalışma esnasında desteği ve katkılarını esirgemeyen danışman hocam Mehmet Ali Altuncu'ya minnettarlığımı ifade etmek isterim. Ayrıca çalışmalarım boyunca bana her zaman destek olan aileme ve arkadaşlarıma teşekkür ederim.

2024

Hatice Akıllı

İÇİNDEKİLER

ÖNSÖZ VE TEŞEKKÜR.....	3
İÇİNDEKİLER	4
ŞEKİLLER DİZİNİ	5
ÖZET.....	6
ABSTRACT	8
GİRİŞ	9
İLGİLİ ÇALIŞMALAR	11
COVID-19 RADIOGRAPHY DATABASE.....	15
KULLANILAN TEKNOJİLER VE YÖNTEMLER.....	16
CNN YÖNTEMLERİ	17
DenseNet	17
ResNet	19
VGG	22
ALEXNET	24
INCEPTIONNET	28
LENET	31
EFFICIENTNET	33
MOBILENET	35
MALZEME VE YÖNTEM	39
BULGULAR VE TARTIŞMA	41
SONUÇLAR VE ÖNERİLER	46
KAYNAKÇA	49
ÖZGEÇMİŞ.....	54

ŞEKİLLER DİZİNİ

DenseNet	17
Şekil 3.1.1.	17
Şekil 3.1.2.	18
ResNet	19
Şekil 3.2.1.	20
Şekil 3.2.2.	21
Şekil 3.2.3.	22
VGG.....	22
Şekil 3.3.1.	23
Şekil 3.3.2.	24
ALEXNET	24
Şekil 3.4.1.	25
Şekil 3.4.2.	27
INCEPTIONNET	28
Şekil 3.5.1.	29
Şekil 3.5.2.	29
Şekil 3.5.3.	30
LENET	31
Şekil 3.6.1.	31
Şekil 3.6.2.	33
EFFICIENTNET	33
Şekil 3.7.1.	34
Şekil 3.7.2.	34
Şekil 3.7.3.	35
MOBILENET	35
Şekil 3.8.1.	36
Şekil 3.8.2.	37
Şekil 3.8.3.	38

SONUÇ GRAFİKLERİ.....	44
Şekil 3.9.1.	44
Şekil 3.9.2.	44
Şekil 3.10.1.	45
Şekil 3.10.2.	45

ÖZET

Koronavirüs hastalığı (Covid-19) binlerce can kaybına neden olan ve dünya çapında milyonlarca insanı enfekte eden bulaşıcı bir hastalıktır. İlk vaka ile Çin’de 2019 yılında karşılaşılmış olup o zamandan beri yaklaşık olarak 2 yıl devam etmiş ve dünya genelinde pandemiye neden olmuştur. Covid-19 semptomları genellikle ateş, öksürük, nefes almakta zorluk, tat ve koku alma duyusunda kayıp şeklinde seyretmiştir. Semptomlar virüsle temastan 1-14 gün içerisinde kendisini göstermekte ve herhangi bir covid-19 pozitif vakası ile en ufak bir temasta, kişinin kendisini 14 gün boyunca izole etmesi gerekmektedir. Çoğu insanda hafif semptomlar görülmesine karşın bazı insanlarda yoğun bakımda tedavi görmesini gerektirecek kadar ileri gitmiştir. Enfekte olan kişilerden yaklaşık olarak beş kişiden birinde herhangi bir belirti görülmemektedir ve bu, hastalığın ne kadar hızlı yayıldığının bir göstergesidir. Bu hıza karşı en iyi çözüm erken teşhisti ve bu noktada yüksek doğrulukla hızlı bir şekilde teşhis koyabilecek yapay zekaya ihtiyaç oldu. Bu makalenin amacı, önceden eğitilmiş derin öğrenme algoritmaları hakkında bilgi sahibi olmak ve daha öncesinde yapılmış çalışmalara bakarak hangi yöntemlerin hangi teknolojiler ile tespit doğruluğunu en üst düzeye çıkardığına bakıp dijital göğüs röntgen görüntülerinden Covid-19 pozitif vakaların otomatik tespiti için bir yöntem belirlemektir.

ABSTRACT

Coronavirus disease (Covid-19) is an infectious disease that has caused thousands of deaths and infected millions of people worldwide. The first case was encountered in China in 2019 and has continued for approximately 2 years since then, causing a pandemic worldwide. Covid-19 symptoms generally include fever, cough, difficulty breathing, and loss of taste and smell. Symptoms appear within 1-14 days of contact with the virus, and any slightest contact with a Covid-19 positive case requires the person to isolate themselves for 14 days. Although most people have mild symptoms, some people have gone so far as to require intensive care treatment. Approximately one in five infected people do not show any symptoms, and this is an indication of how quickly the disease spreads. The best solution to this speed was early diagnosis, and at this point, artificial intelligence that could diagnose quickly with high accuracy was needed. The aim of this article is to gain knowledge about pre-trained deep learning algorithms and to determine a method for automatic detection of Covid-19 positive cases from digital chest X-ray images by looking at which methods and technologies maximize detection accuracy by looking at previous studies.

GİRİŞ

Koronavirüs hastalığı (Covid-19) son derece bulaşıcı bir hastalıktır ve Dünya Sağlık Örgütü (WHO) tarafından 11 Mart 2020'de pandemi olarak ilan edilmiştir. Hızlı bulaşıcılığa sahip olması ve dünyadaki insanların neredeyse ilk defa böyle bir durumla karşılaşması sebebiyle bilinçsiz ve deneyimsiz olmaları, milyonlarca insana bulaşarak en başta sağlık ve daha birçok sektörde benzersiz zorluklara yol açmıştır. Özellikle erken teşhis ve hızlı müdahale, hastalık yayılımını kontrol altına almanın en kritik adımları arasında yer almaktaydı [1].

Covid-19 hastalığını kapalı insanların çoğu hafif veya orta şiddette hastalığı geçirirken, bazıları ölümcül bir zatürre geliştirdi ve bazılarında birkaç kalıcı ufak rahatsızlıklar bile bıraktı. Öncesinde diyabet, solunum yolu hastalıkları veya kanser gibi rahatsızlıklara sahip kişilerde ve yaşlı insanlarda ciddi hastalık geliştirme olasılığının daha yüksek olduğu görüldü. Dünya genelinde 700 milyona yakın insanın bu hastalığı geçirdiği ve 7 milyon civarında insanın hayatına mal olduğu bilinmektedir [2].

Medikal görüntüleme, akciğer hastalıklarının tespiti için yaygın olarak kullanılırsa da pandeminin hızına karşı doğru bir şekilde analiz yapması zorlayıcıydı. Covid-19 tespitinde kullanılan PCR kitinin yaklaşık 120-130 dolar ettiği ve PCR makinesi için özel bir biyogüvenlik laboratuvarı gerektirdiği bunun da 15.000 ila 90.000 doları arasında değişkenlik gösterdiği göz önüne alındığında böylesine pahalı bir yöntem, birçok ülke için hastalığın hızla yayılmasına yol açarak durumu daha da kötü hale getirmiştir. Ayrıca radyologların Covid-19 ve viral zatürre hastalığını ayırt etmesi zordu. Bundan dolayıdır ki radyolojik görüntülerin işlenmesi ve analizi konusunda yeni teknikler, yeni yöntemler geliştiren yapay zekaya başvurulmuştur. Yapay zeka birçok tıbbi alanda zaten kullanılmaktaydı. Mesela meme kanseri, beyin tümörü, genetik hastalıkların ve kalple ilgili birçok rahatsızlık teşhisinde umut vadeden sonuçlar vermektedir. Özellikle CNN yöntemleri medikal görüntülerden anlamlı özellikler çıkararak Covid-19 ve viral zatürre gibi akciğer hastalıklarının teşhisinde ve sınıflandırmada güzel sonuçlar yaratmakta ve işi kolaylaştırmaktadır [1].

Bu makalede COVID-19 Radiography Database açık kaynaklı veri kümesi ele alınarak, çeşitli çalışmalarda AI tabanlı modellerin Covid-19 ve viral pnömoni teşhisindeki performansları ele alınmıştır. Çeşitli makaleler incelenmiş, veri ön işleme ve model geliştirme adımları göz önünde bulundurarak nasıl sınıflandırılmış, hangi CNN yöntemleri kullanılmış ve bu CNN yöntemlerinden nasıl bir doğruluk ve verimlilik elde edilmiş, CNN yöntemleri dışında nasıl farklı teknikler kullanılmış ve bu yeni tekniklerin Covid-19 ve viral pnömoni teşhisindeki sonuçlara katkısı nasıl olmuş değerlendirilmiştir.

En sık kullanılan CNN yöntemlerini detaylı bir şekilde incelediğim bir bölüm bulunmaktadır. Burada Covid-19 teşhisi üzerine yapılan çalışmalarda kullanılan en çok yöntemleri analiz ettim. İncelediğim CNN yöntemlerinde; kullanıldığı alanları, detaylı şekilde mimarisini, mimarisinin adımları (katmanları) ile bu adımların pozitif olarak kattıkları ve negatif olarak açabileceği yolları, diğer mimarilerden farkının ne olduğu ve bu farkın sonuca negatif veya pozitif etkilerini, birbirleri ile kıyaslandığında verdikleri performans ve hangi durumda hangi CNN yöntemini kullanmanın daha avantajlı olabileceğinden, bu yöntemlerin kısaca tarihçesinden ve şu anki mevcut durumu ile gelecekteki potansiyel uygulama alanlarını, varsa farklı sürümlerini ve bu sürümleri arasındaki benzerliklerle farklılıkları görsellerle destekleyerek aktardım .

Ayrıca, yapay zekâ tabanlı yaklaşımların geleneksel yöntemlere kıyasla sağladığı hız, doğruluk, verimlilik ve ölçeklenebilirlik avantajlarına odaklanılmaktadır. Bu bağlamda; yapay zeka ve derin öğrenme çözümlerinin, yalnızca sıkıntılı bir süreç olan pandemi koşullarında değil, gelecekteki benzer sağlık ve birçok alan krizlerinde de etkili bir araç olabileceği vurgulanmaktadır.

İLGİLİ ÇALIŞMALAR

Muhammed ve arkadaşları (2024), covid-19 tespiti için 8 farklı önceden eğitilmiş CNN modelini (Inceptionv3, ResNet101, CheXNet, VGG19, DenseNet201, MobileNetv2, SqueezeNet, ResNet18) eğitti, doğruladı ve test ettiler. İki farklı yöntemle eğitim görüntü artırma ile ve görüntü artırma olmadan COVID-19 ve normal görüntüleri sınıflandırdılar. Görüntü artırma kullanıldığında DenseNet201'in, görüntü artırma kullanılmadığında ise CheXNet'in diğer ağlardan daha iyi performans gösterdiği görülmüştür ve çok yüksek başarı oranı sağlanmıştır [3].

Alex ve arkadaşları (2022), elektrokardiyogram (EKG) iz görüntülerinden covid-19 tespiti için 6 farklı CNN modeli (ResNet18, ResNet50, ResNet101, InceptionV3, DenseNet201 ve MobileNetv2) iki sınıflı sınıflandırma (normal ve COVID-19); üç sınıflı sınıflandırma (normal, COVID-19 ve diğer CVD'ler) ve son olarak beş sınıflı sınıflandırma (normal, COVID-19, MI, AHB ve RMI) için kullanmışlardır. İki sınıflı ve üç sınıflı sınıflandırma için Densenet201, beş sınıflı sınıflandırma için ise InceptionV3 yüksek başarı oranı sağlamıştır [4].

Aman ve arkadaşları (2020), covid-19 analizi ve teşhisi için 14 farklı CNN modelini kullanmışlardır. Bunların arasında VGG, ResNet, DenseNet, MobileNet, NasNet, InceptionV3 ve Xception bulunmaktadır. Hepsinin yüksek başarı oranı sağlamanın yanında bunların arasında DenseNet121, DenseNet169 ve MobileNetV2 çok yüksek başarı oranı sağlamaktadır [5].

Aras ve arkadaşları (2021), covid-19 tespiti için ResNet18, ResNet50, ResNet101, VGG16 ve VGG19 modelleri kullanarak derin özelliklerin eğitimi için Doğrusal, İkinci Dereceden, Kübik ve Gauss olmak üzere çeşitli çekirdek fonksiyonlarıyla SVM sınıflandırıcısı kullanmışlardır. Çok yüksek başarı oranı Kübik çekirdek sınıflandırmada sağlanmıştır [6].

Sejuti ve arkadaşları (2021), covid-19 tespitindeki zorlukları ele almış ve daha hızlı, daha doğru tanı konması için 315 derin öğrenme modelini kapsamlı bir şekilde analiz etmişlerdir. Veri dengesizliğini aşmak amacıyla veri artırma, karar ağaçları ve rastgele ormanlar tekniklerini kullanma gibi çeşitli yöntemler kullanmışlardır. Sonuç olarak ise DenseNet201 modelinin Quadratic SVM sınıflandırıcısı ile birleşerek çok yüksek performans gösterdiği görülmüştür [7].

Pangoth ve arkadaşları (2023), covid-19 tespiti için ilk olarak histogram eşitlemesi ve veri artımı yapmışlardır. Ardından sınıflandırma için CNN modellerine göre görüntülerdeki desenleri yakalamada daha iyi olan Vision Transformer (ViT) modelini kullanmışlardır. GradCAM tekniğinden lezyonu tahmin etmek için kullanılan girdi bölgesini görselleştirmek için yardım almıştır ve yüksek başarı oranı yakalamıştır [8].

Rubina ve arkadaşları (2022), covid-19 tespiti için CNN'leri (VGG16, InceptionV3, Xception) iki sınıflandırma senaryosunda öğrenme tabanlı transfer modellerini karşılaştırmışlardır. Senaryo 1: normal/covid-19 sınıflandırmasından oluşur ve Senaryo 2: normal/covid-19/pnömoni sınıflandırmasından oluşur. Sonuç olarak VGG16 kullanarak ikili sınıfı %100 doğruluk ve üçlü sınıflandırmada %93,75 doğruluk elde etmişlerdir [9].

Weiguang ve arkadaşları (2023), covid-19 tespiti için normal görüntüler ile arasındaki farkları ayırt etmek için fraktal boyut ve quadtree ölçütleri kullanarak yüksek seviyeli sınıflandırma tekniği ile görüntüleri sınıflandırmışlardır. Bunu da yarıçap komşuları (Radius Neighbors) ve k-NN (k-Nearest Neighbors) algoritmalarının birleşimi ile normal ve COVID-19 olarak iki farklı sınıf için oluşturulan ayrı ağlar tarafından temsil edilen noktalarla eşleştirilmesinde kullanarak sağlamışlardır. Bunun sonucunda iyi sonuçlar elde etmiş ve yüksek doğruluk oranları sağlamışlardır [10].

Oleksandr ve arkadaşları (2024), covid-19 tespiti üzerinde veri artırmanın etkisini araştırmak için 3 veri artırma yaklaşımını (hiç artırma yok, klasik artırma ve GAN tabanlı artırma) iki veri kümesiyle Inception-v3 modelini eğiterek karşılaştırmışlardır. İlk veri kümesi 4 sınıf başına 1000 görüntü, ikincisi veri kümesi 4 sınıf başına 500 görüntü içermektedir. Eğitim tamamlandıktan sonra CNN için GAN destekli eğitim veri seti, küçük veri seti için sınıf başına 2000 görüntü ve mikro veri seti için sınıf

başına 1500 görüntü içeriyordu. Test sonucunda herhangi bir artırma yaklaşımının hiç artırma olmamasından ve küçük veri kümeleriyle büyük veri kümelerine göre daha önemli bir rol oynadığı daha iyi olduğu görüldü [11].

Ameer ve arkadaşları (2022), covid-19 tespiti için öncelikle çoklu filtre tekniği ile kontrast iyileştirmesi yapmışlardır. Daha sonrasında önceden eğitilmiş model olan EfficientNet-B0 ve MobileNet-V2, Bayesian Optimizasyon kullanılarak optimize etmişlerdir. Özellik çıkarımı sonrasında slicing-based serial fusion yöntemiyle bu özellikleri birleştirmiş ve sınıflandırma işlemi yapılmışlardır. Görselleştirme için Grad-CAM tekniğinden yararlanılmıştır. Yapılan testler sonucu çok yüksek başarı oranı sağlanmıştır [12].

Primanshu ve arkadaşları (2022), covid-19 tespiti için eldeki verileri 4 gruba (covid, normal, akciğer opaklığı, viral zatürre) ayırmışlardır. Görüntülerden eğitim, test ve doğrulama olmak üzere üç bağımsız veri kümesi oluşturmuşlardır. Küçük bir kısım, eğitilen modelin etkinliğini değerlendirmek için bir doğrulama kümesi olarak saklanırken, geri kalanı beş kata katlanır. Eğitim seti verilerini NasNet, VGG19, ResNet50, DenseNet121 ve Xception modellerini eğitmek için kullanmışlar ve ResNet50 çok yüksek başarı oranı sağladığını görmüşlerdir [13].

Mominul ve arkadaşları (2021), covid-19 tespiti için ilk olarak MADF tekniği ile gürültü azaltmışlardır. Daha sonrasında HOG tekniği, X-ışını covid-19 veri setinden bir özellik vektörü çıkarmak için kullanıldılar. Aynı görüntülerden başka bir özellik vektörü çıkarmak için VGG19 modeli kullanıldılar ve bu iki özellik birleştirildi, sınıflandırma modelini eğitmek için girdi olarak kullanıldı. Havza segmentasyonu ile hasarlı bölgeler işaretlendi ve ardından sistemin performansını daha ileri düzeyde tutmak için genelleme ve 5-katlı doğrulama teknikleri uygulandı. Bunun sonucunda çok yüksek başarı oranı sağlanmıştır [14].

Shervin ve arkadaşları (2020), covid-19 tespiti için eldeki 2000 eğitim verisini çeşitli şekillerde dönüştürerek 4 farklı (ResNet18, ResNet50, SqueezeNet ve DenseNet-121) eğitilmiş modelin performansını karşılaştırmışlardır. Geriye kalan 3000 test verisinde ise üçlü sınıflandırma sonucunda SqueezeNet diğerlerine göre yüksek başarı oranı sağlamıştır [15].

Mustafa ve arkadaşları (2021), covid-19 tespiti için veri gizliliği ve yüksek hesaplama gücü sebebiyle federasyonlu makine öğrenimini ve geleneksel makine öğrenimini karşılaştırmışlardır. Keras ve TensorFlow kullanarak hem ikili (covid ve zatürre) hem de üçlü (covid, zatürre ve normal) sınıflandırmada federasyon modeli daha yüksek tahmin doğruluğuna sahip çıkmıştır [16].

COVID-19 RADIOGRAPHY DATABASE

Çeşitli kliniklerden veya üniversitelerden toplanan Covid-19 pozitif vakalar, normal ve viral zatürre göğüs röntgeni (CXR) görüntülerinden oluşan bir veri tabanıdır. Bu veri tabanını oluşturmaktaki en büyük amaç, Covid-19 pozitif hastaların büyük bir hızla artması fakat doktorların sayısal olarak aynı hızda artamadıklarından dolayı bu hastalığı teşhis etmede aynı performansı gösterememesidir. Bundan dolayı Covid-19 pozitif vakaların erken teşhisi için yapay zeka ve derin öğrenme modellerini eğitmek için kullanıldı.

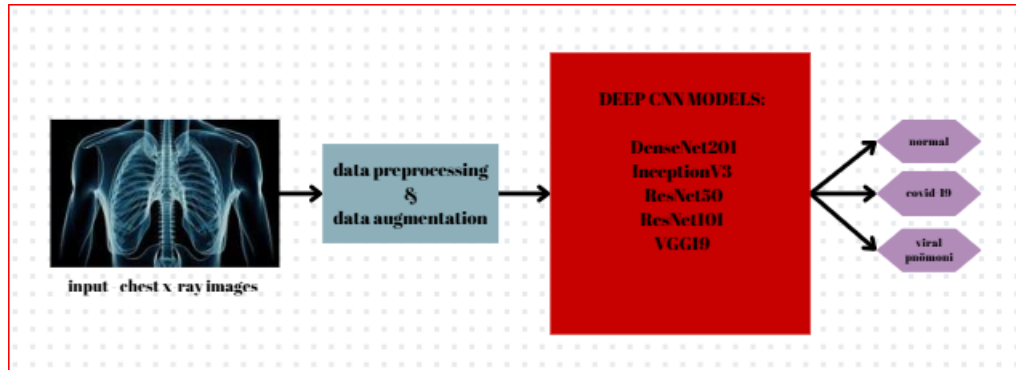
Bu veri tabanı aşamalar halinde yayınlanmıştır. İlk aşama; 219 Covid-19, 1341 normal ve 1345 viral zatürre göğüs röntgen görüntüsü içermekteydi. Daha sonrasında ilk güncelleme ile Covid-19 sınıfı 1200 görüntüye çıkmış olup, ikinci güncelleme ile Covid-19 sınıfı 3616, normal sınıf 10.192, viral zatürre sınıfı 1345 ve akciğer opaklığı (akciğer enfeksiyonu) sınıfı 6012 görüntüye çıkarılmıştır. Sınıflandırma sonucunda ise görüntüler 3 gruba ayrılır: Covid-19 pozitif vakalar, normal vakalar ve zatürreli vakalar.

Bu veri seti çok fazla gri tonlamalı görüntü içermektedir ve her görüntü ait olduğu sınıfa dair bir etiket içermektedir fakat bu görüntülerin dağılımı ne kadar dengeli yapılmaya çalışılsa da Covid-19 sınıfının görüntüleri diğerlerine kıyasla daha azdır. Bu da modelimizin eğitim aşamasını negatif olarak etkilemektedir. Modelimizin eğitim aşamasını negatif olarak etkileyecek bir diğer unsur ise Covid-19 ve viral zatürre görüntülerinin arasındaki benzerliktir, model bu iki sınıfı ayırt etmekte zorlanabilir. Ayrıca mevcut görüntülerin kalitesi yüksek olmasına rağmen birbirlerine kıyasla kontrast farklılıkları içermektedir. Bu da görüntülere bir ön işleme adımı gerektirir.

Veri kümesi eğitim, doğrulama ve test olmak üzere üçe ayrılır. Yapılan çalışmalarda genellikle eğitim kümesi için %80, doğrulama kümesi için %10 ve test kümesi için %10'u kullanılmıştır.

KULLANILAN TEKNOJİLER VE YÖNTEMLER

COVID-19 Radiography Database üzerinde yapılan çalışmalarda, derin öğrenmede en çok tercih edilen dil olan Phtyon kullanılmıştır. Geniş kütüphane desteği sayesinde veri işleme ve model geliştirmede büyük kolaylık sağlar. Röntgen görüntüleri üzerinde ölçeklendirme, filtreleme, iyileştirme ve veri artımı gibi birçok görüntü işlemleri için OpenCV, Pillow ve Keras açık kaynaklı kütüphaneleri tercih edilmiş olup UNet'ten de destek almıştır. Özellik çıkarımı için TensorFlow ve NumPy kütüphaneleri kullanılmıştır. Model geliştirme sürecinde çeşitli CNN yöntemleri (DenseNet, ResNet, VGGNet, EfficientNet, InceptionNet...) kullanılmaktadır. Bu süreçte Google Colab, Kaggle Notebooks, TenserFlow ve PyTorch yardımcı olmuştur. Model dağıtımı için Docker, modelleri web uygulamalarında kullanılabilir hale getirmek için FastAPI ve modellerin bulut üzerinde barındırılması için de Google Cloud kullanılmıştır.

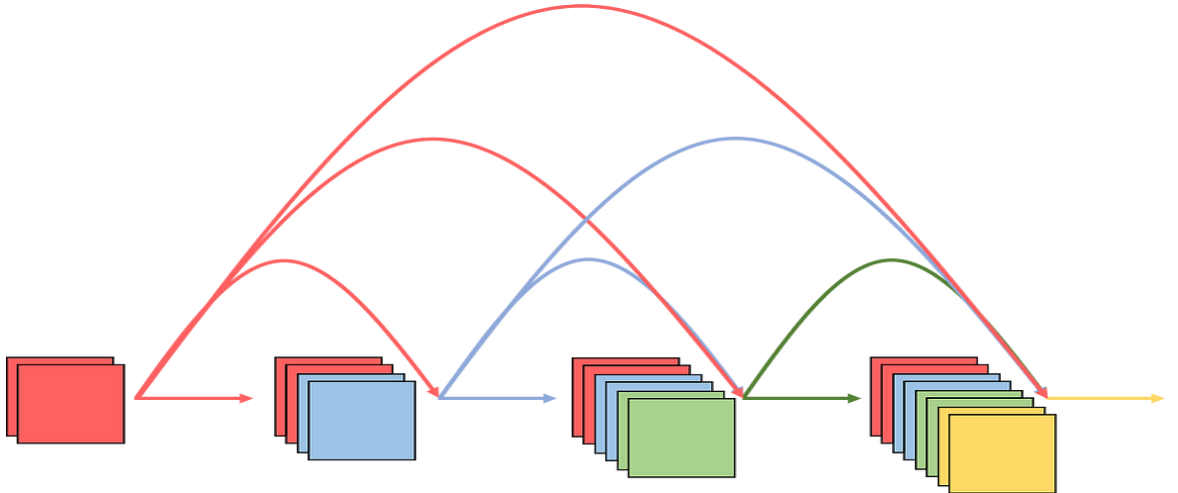


CNN YÖNTEMLERİ

DenseNet

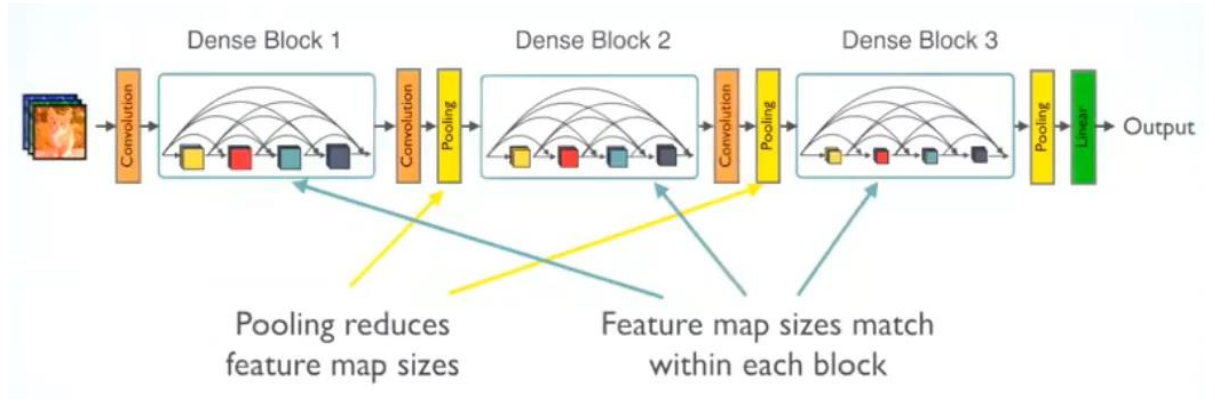
DenseNet, derin öğrenme alanında görüntü işleme ve sınıflandırmada kullanılır dolayısıyla bu özellikle tıbbi alanda çeşitli hastalıkların tespitinde çok fazla tercih edilmektedir. Ses sinyali tespiti ve gürültü azaltmada da iyi performans gösterdiğinden günlük hayatın içinden trafik alanında da tercih edilir.

DenseNet mimarisi; diğer mimarilerden farklı olarak bir önceki çıktıyı, o katmandan sonraki tüm katmanlara girdi olarak alırken atlama bağlantılarını artırır ve yoğun bir bağlantı sayısı ile bilgi akışı oluşmuş olur. Çünkü her katman önceki tüm katmanların özellik haritalarına erişebilir duruma gelir [17]. Bundan dolayı overfitting durumuna pek düşmez. Bunun dışında diğer CNN yöntemlerinde bilgi ve eğitimlerin tüm katmanlar arasında etkili bir şekilde yayılmakta zorlanması “kaybolan eğitim sorunu” (vanishing gradient problem) olarak adlandırılır [18]. Bu sorun, bir derin öğrenme modelinin eğitilmesi sırasında geri yayılımda (backpropagation) weight ve bias’daki ciddi değişimlerin sonuca etki etmemesi nedeniyle öğrenmenin efektif bir şekilde gerçekleşmemesiyle ortaya çıkar.



Şekil 3.1.1. DenseNet bağlantısı [15]

Varsayalım ki L sayıda katmanımız var, diğer geleneksel CNN yöntemlerinde katmanlar arasında L bağlantı olacaktır ancak DenseNet mimarisinde, yaklaşık olarak $L(L+1)/2$ sayıda bağlantımız olacaktır [18]. Yani yoğun bir ağda, diğer modellerden daha az sayıda katmana sahip olarak kendi modelimizi daha kolay bir şekilde eğitebiliyor olacağız. Yoğun blok ve katmanların oluşmuş olması bunun gibi bazı avantajlara sahipken bazı dezavantajlara da sahiptir. Bir diğer avantajı ise her katman daha az sayıda yeni bilgi öğrenmek durumunda kalır ve bu da ağın parametre karışıklığını önlemiş olduğundan dolayı parametre verimliliği yüksek olur. Dezavantajı ise her katman kendinden önceki bütün katmanların çıktılarını girdi olarak kullanması gerektirdiğinden bellek kullanımını artırır ve yoğun bağlantılardan dolayı modelin karmaşıklığı da vardır.



Şekil 3.1.2. DenseNet mimarisi [17]

DenseNet, yoğun blok (dense block) olarak adlandırılan 3 ardışık işlemten oluşur. Bu ardışık 3 işlem: toplu normalizasyon (Batch Normalization), ardından düzeltilmiş doğrusal bir birim (ReLU) ve ardından 3x3 evrişim işlemidir.

Batch Normalization, öğrenmenin hızlanması amacıyla ağıdaki katmanın, önceki katmanın öğrenmesini beklemek zorunda kalmayarak eş zamanlı öğrenimi sağlar.

ReLU, pozitif değerlerde aktif olan bir aktivasyon fonksiyonudur bundan dolayı ara katmanlardaki herhangi bir negatif çıkışı aktif etmeyecektir. Bu da sistemin hızlı ve verimli çalışma konusundaki performansını olumlu anlamda etkileyecektir.

Her katman çıktı sayısından çok daha fazla girdi sayısına sahiptir. Bundan dolayı giriş özellik haritalarının sayısını (kanal sayısını) azaltmak için her 3x3 evrişim işleminden önce 1x1 evrişim işleminden geçer. Bu da bottleneck layers olarak adlandırılır. Bu işlemin ardından 2x2 ortalama havuzlama katmanına sokulur çünkü ağ birden fazla yoğun bağlantılı yoğun bloğa bölünür ki alt örneklemeyi kolaylaşsın. Bu aynı zamanda sıkıştırma (compression) işlemi olarak geçer ve modelin karmaşıklaşmasını da önlemeye çalışılmış olur [18].

ResNet

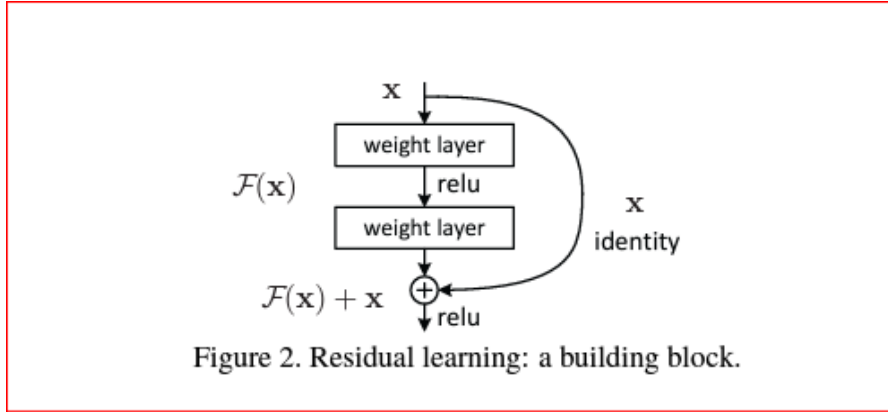
ImageNet yarışmasında diğer mimarilerden daha derin ağ modeline fakat daha az karmaşıklığa sahip olan 152 katmana kadar derinliğe sahip, DenseNet'in de temelini oluşturan ResNet mimarisi %3,57 hata oranı ile ILSVRC 2015 sınıflandırma görevinde 1. olmuştur [19].

Daha derin ağların problemi, ağ derinliğinin artmasıyla doğruluğun doygunluğa ulaşması ve ardından hemen hızla düşmesiydi bu da kaybolan gradyan sorunu idi fakat bu geri yayılım ile büyük ölçüde önlenmiş oldu. ResNet'te daha spesifik olarak, ağdaki belirli katmanları atlmasına izin veren “artık bağlantılar (atlama bağlantıları)” bu sorunu çözer.

ResNet, belirli bir sırayla residual blokların bir araya getirilmesi ile oluşturulmuş bir ağıdır. Residual bloklar, iki aktivasyon fonksiyonu (ReLU) arasında kalan evrişim bloklarını atlar.

$H(x)=F(x) + x$ olsun. x : girdi [19].

Yani geleneksel bir ağ modelinde x girişimiz ardışık katmanlardan geçerek sonucunda $H(x)$ çıktı fonksiyonu ile bir öğrenme gerçekleşirken residual blokta $H(x)-x$ yani artık fonksiyon dediğimiz $F(x)$ 'i öğrenir. Girişimiz olan x , doğrudan katmanların çıktı fonksiyonuna eklenmiş olur. Bu da özellikle çok derin ağa sahip mimarilerde daha verimli öğrenmeyi ve gelişmiş performansı sağlar.



Şekil 3.2.1. Residual Layer (blok)[20]

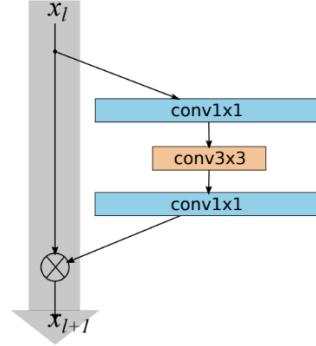
ResNet mimarisinde ilk katman 7x7 konvolüsyon ve max pooling katmanlarıdır. Boyutu yarıya indirmek için stride (adım) sayısı genellikle 2 olarak ayarlanan bu katmanlar, girdiyi küçültür ve özellik haritalarını çıkarır.

Bu katmanlardan sonra residual bloklar gelir ve ağı derinlik seviyelerinin farklılığına göre iki tipte residual bloklarıyla karşılaşırsınız.

Daha basit ResNet mimarilerindeki (mesela ResNet-18 ve bu mimari 8 adet residual blok içerir) bloklar her biri aynı çıkış boyutuna sahip iki ardışık konvolüsyon katmanından oluşurlar. İlk konvolüsyon katmanı girdiyi 3×3 boyutunda filtreler, burada stride genellikle 1'dir ve son olarak çıktıya Batch Normalization ile ReLU işlemleri uygulanır. İkinci konvolüsyon katmanı aynı şekilde girdiyi 3×3 boyutunda filtreler, burada da stride genellikle 1'dir ve son olarak çıktıya sadece Batch Normalization işlemi uygulanır. Yani ReLU uygulanmaz. Bu işlemlerden sonra girdimiz olan x , konvolüsyon katmanlarının çıktılarına eklenir ve ardından ReLU aktivasyon fonksiyonu uygulanır. Bu bir adet residual bloğun yaptığı işlemdir [19], [20].

Daha kompleks ResNet mimarilerinde (mesela ResNet-50 ve bu mimari 16 adet residual blok içerir) ise residual bloklar 3 katmandan oluşan bir bottleneck bloğuna (yapısına) sahiptir. İki adet (3x3) evrişim kullanmak yerine sırasıyla (1x1), (3x3), (1x1) şeklinde konvolüsyon katmanları kullanır. Bu işlemde sırasıyla boyut azaltma yapılır ki hesaplama yükü düşsün, temel işlemlerin yapıldığı katman ve boyutu tekrar artırırız

ki orijinal giriş seviyesine geçsin. En son girdi x eklenir ve sonuç ReLU aktivasyon fonksiyonundan geçirilir [19].



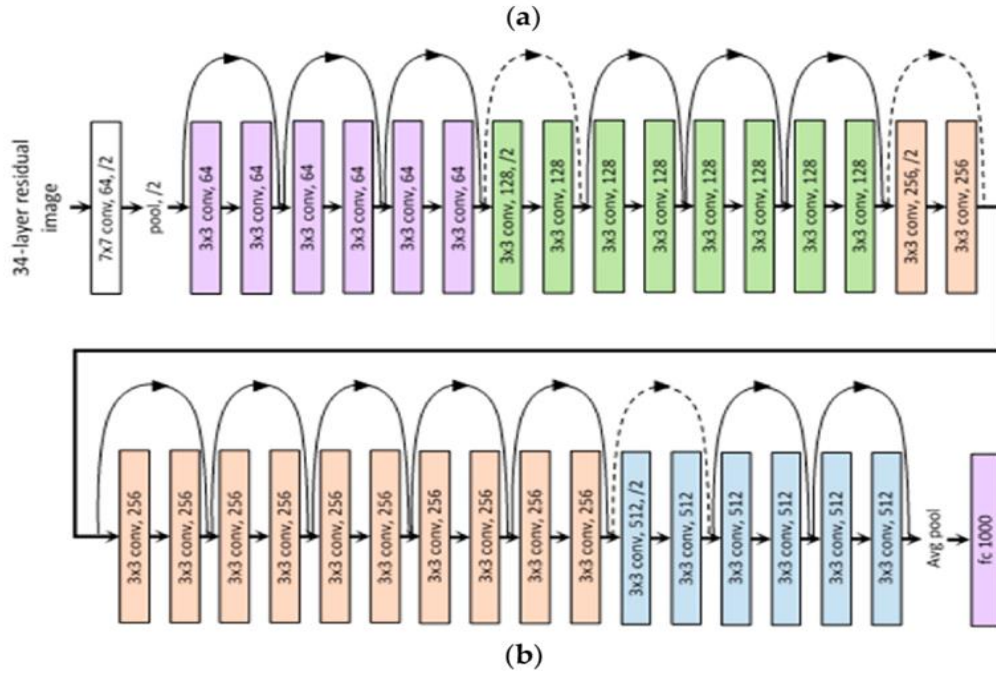
Şekil 3.2.2. Bottleneck (darboğaz) yapısı

[19]

Bottleneck, parametre ve maliyet düşürülmesine yarar, hesaplama verimliliği de artmış olur. Böylece daha derin ağların eğitilmesi mümkün hale gelmiş olur.

ResNet mimarisi derinleştikçe daha karmaşık özellikleri öğrenmek için ve herhangi bir residual blokta girişin ve çıktının boyutları uyuşmuyorsa bazı bloklarda stride 2 kullanılır. Bu da downsampling adıdır.

Son katman olan fully connected (tam bağlantılı) katmanı çıkarılan özelliklerin sınıflandırmaya girdiği katmandır. Residual bloklarından gelen çıktılar average pooling (GAP) katmanına verilmesiyle tüm özellik haritalarının ortalaması alınır ve çıkış sabit bir uzunluğa dönüştürülmüş olur. Bu çıkış birimlerinin sayısı, sınıf sayısına da eşittir (Görseldeki gibi ImageNet için 1000 sınıf) [19].



Şekil 3.2.3. ResNet-34 katmanlı mimari[21]

Böylelikle ResNet mimarisi, derin ağların eğitiminde yüksek bir performans sağlamış olur. Çünkü residual bloklar, gradyanların kaybolmasını önler ve kolay bir tasarıma sahiptir. Tıbbi alanda görüntü sınıflandırmada çok fazla tercih edilen bir yöntemdir. Bunun yanında çok fazla derinlikteki ağlarda entegrasi zor olabileceğinden fazla bellek alanı kullanacak ve eğitim süresi uzayabilecektir [19].

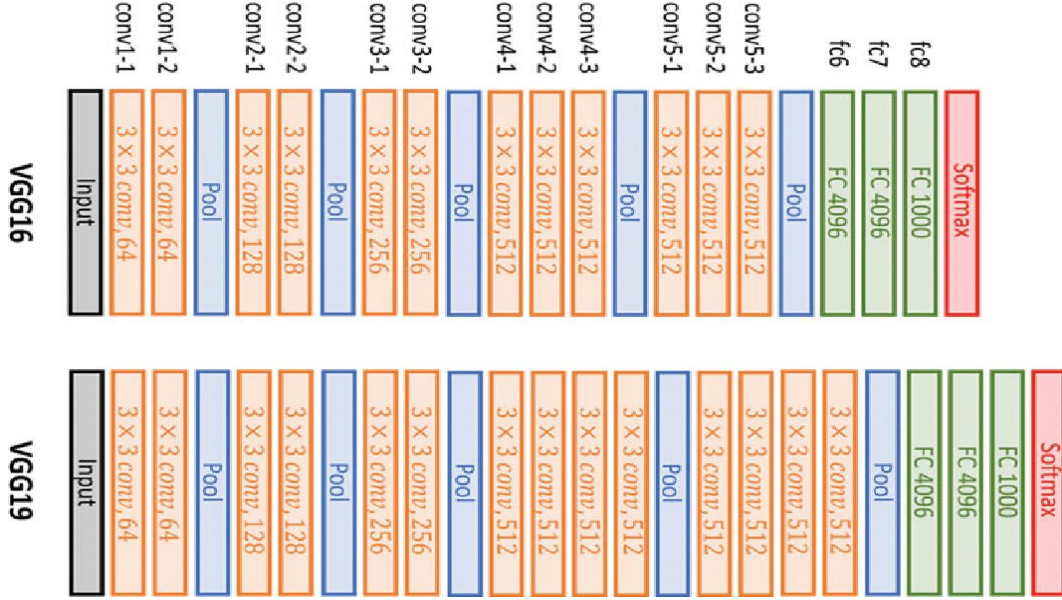
VGG

Derin öğrenme ve bilgisayarlı görme alanlarında önemli bir yere sahip olan ve “Görsel Geometri Grubu” anlamına gelen VGG (Visual Geometry Group), oldukça sade bir ağ mimarisine sahip olup derinlik artışıyla model performansını iyileştirmede güzel sonuçlar elde eder. ImageNet’te, %92,7 doğruluk oranı yakalayan VGG16 modeli, en iyi 5 test doğruluğuna ulaşarak AlexNet’i geride bırakmıştır [22].

Bu modelin temeli, daha iyi performans görmek amacıyla daha derin ağlarda çalışılma yapılması gerektiğinden fakat bunun da aşırı öğrenme olarak bildiğimiz overfitting’e yol açtığı gözlemlendikten sonra atılmıştır. Bundan dolayı daha küçük filtreler kullanarak ağın derinliği arttırıldığında ise istenen sonuca varılmıştır.

Çoklu katmana sahiptir ve VGG-16 (13 konvolüsyon -convolutional- katmanı + 3 tam bağlantılı -fully connected- katman) veya VGG-19 (16 konvolüsyon katmanı + 3 tam

bağlantılı katman) modellerindeki sayılar evrimsel katman sayısını verir ve buradaki konvolüsyonel katmanları ve max pooling katmanlarını ardışık olarak kullanır [23].



Şekil 3.3.1. VGG16 ve VGG19 mimarisi [24]

Eğitim sırasında giriş olarak, 224×224 RGB görüntüleri alır. Görüntü, çok küçük bir alıcı alana sahip 3x3 filtrelerden geçirilir. Bu adım konvolüsyonel katmanında gerçekleşir ve derinliği artırabilmemiz için en önemli adımdır çünkü bu filtreler çok fazla derinlere inmeden daha fazla bilgi çıkarımı yapılmasını sağlıyor [22]. Her konvolüsyonel katman aynı zamanda ReLU aktivasyon fonksiyonunu kullanarak eğitim süresini kısaltır. Çünkü ReLU; giriş pozitifse girişi çıktı olarak veren, negatifse çıktıyı 0 veren bir parça parça doğrusal olan fonksiyondur. Bu katmanda stride 1'dir.

VGG mimarisinde her iki veya üç konvolüsyonel katmandan sonra maksimum havuzlama (max pooling) katmanı yer alır. Bu katman 2x2 boyutunda uygulanarak veri boyutunu küçültür. Stride 2 olarak kullanılır ki örneğin 224×224 çözünürlüğe sahip bir giriş, maksimum havuzlama katmanından sonra 112×112 boyutuna indirgenmiş olur. Böylelikle hesaplama tasarrufu sağlanmış olur ve bu adım sadece maksimum havuzlamada uygulanır. Bu da VGG'ye ait bir belirgin özelliktir [22].

En son ise tam bağlantılı katman (fully connected layer) bulunur. Genellikle 3 adet tam bağlantı katmanı içerir ve bu katmanların ilk ikisi 4096 kanala sahipken, üçüncü yani son katmanda 1000 kanal bulunur.”Bu katmanlar, modelin öğrenilen özellikleri sınıflandırma aşamasına dönüştürmesini sağlar.”

Bunlardan sonra genellikle softmax aktivasyon fonksiyonu kullanılarak son sınıf tahminini yapar.

Görüldüğü üzere VGG son derece basit ve anlaşılabilir bir yapıya sahiptir. Yapılan çalışmalara göre VGGNet derinlik arttıkça özellik çıkarımında daha iyi performans ve doğruluk oranı göstermektedir.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Şekil 3.3.2. VGGNet karşılaştırması sonuçları [22]

Şekildeki gibi farklı katmanlara sahip ağırları birbirinden bağımsız olarak karşılaştırmışlardır. İşte elde edilen bazı sonuçlar:

- A-LRN ağı kullanılmasının, herhangi bir normalleştirme katmanı olmadan A modelinde iyileştirme yapmadığı gözlemlendiği için diğer mimarilerde (B-E) normalleştirme kullanmamışlardır [23].
- Derinliğin artmasıyla birlikte sınıflandırma hatası azalmış; küçük filtrelere sahip ağırların, büyük filtrelere sahip ağırlardan çok daha iyi performans göstermiştir. En iyi sonucu E (VGG19) yapılandırmasından almışlardır [23].

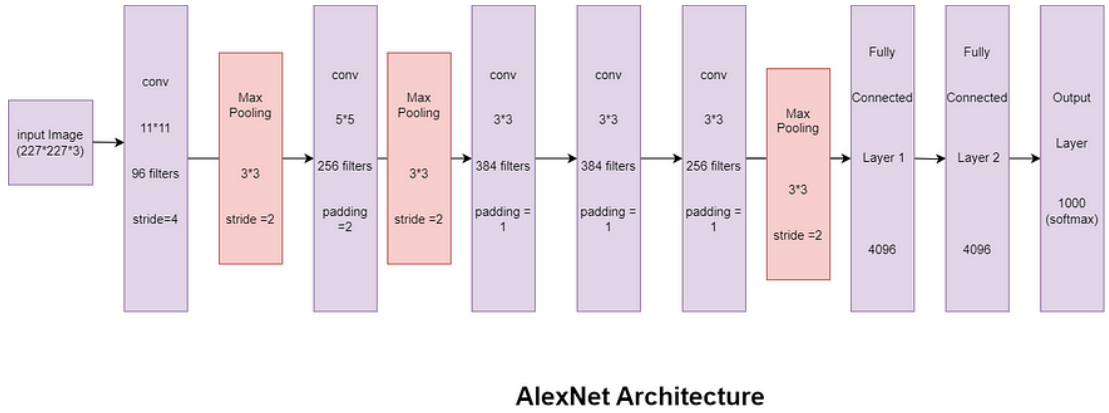
ALEXNET

2012 yılındaki ImageNet görsel tanıma yarışmasında, ImageNet veri kümesindeki sınıflandırma görevinde %16 hata oranı ile birinci olmuştur. Bu model, yapay sinir ağılarının görsel tanıma gibi derin öğrenme görevlerinde ne kadar etkili olabileceğini

göstermiştir. AlexNet, bir CNN yöntemidir ve girdi olarak görsel veri alıp bu veriyi çeşitli katmanlardan geçirdikten sonra özelliklerine göre sınıflandırmak için kullanır [25].

Diğer mimarilerden farklı birkaç farklı teknik kullanır. Mesela eğitim performansını artırmak için GPU kullanan ilk mimaridir. Ayrıca fully connected katmanlarında dropout gibi regularizasyon tekniklerini de kullanarak modelin overfitting yapmasını engellemekte dolayısıyla da daha iyi bir performans göstermektedir. Overfitting engellemek için aynı zamanda veri artırımına da gitti. Yansıtma ile artırma, görüntülerin rastgele kırılmasıyla artırma veya görüntüyü yatay çevirme gibi çeşitli şekillerde veri artırmıştır. Aktivasyon fonksiyonu olarak ReLU kullandı çünkü diğer aktivasyon fonksiyonlarına göre daha hızlı eğitilebiliyordu [26], [27].

60 milyondan fazla parametreye sahip AlexNet, 5 evrimsel katman, 3 maksimum havuzlama katmanı, 2 tam bağlı katman ve 1 SoftMax katmanından oluşur [27].



Şekil 3.4.1. AlexNet mimarisi [27]

Giriş boyutu 227x227x3 olan RGB görüntü alınır.

1. Katman: Her biri 11×11 boyutunda olan 96 filtreye sahip bir konvolüsyonel katmandır (convolutional layer). Stride 4'tür. Dolgu yoktur. ReLU aktivasyonu kullanılır. Çıktı = $((227 + 2*0 - 11)/4) + 1 = 55$ olur. Çıkış özellik haritası da 55x55x96 olur.

Bir evrişim katmanının çıktı boyutunu hesaplamak için şu formül kullanılır :

$$\text{output} = \frac{(\text{input size} + 2 * \text{dolgu} - \text{filter size})}{\text{stride}} + 1$$

2. Katman: 3x3 filtre boyutunda, 2 adımlı maksimum havuzlama katmanı uygulanır. Dolgu yoktur. Çıktı = $((55 + 2*0 - 3)/2) + 1 = 27$ olur. Çıkış özellik haritası da 27x27x96 olur.

3. Katman: Her biri 5x5 boyutunda olan 256 filtreye sahip bir konvolüsyonel katmanıdır. Stride (adım sayısı) 1'dir. Dolgu 2'ye eşittir. ReLU aktivasyonu kullanılır. Çıktı = $((27 + 2*2 - 5)/1) + 1 = 27$ olur. Çıkış özellik haritası da 27x27x256 olur.

4. Katman: 3x3 filtre boyutunda, 2 adımlı maksimum havuzlama katmanı uygulanır. Dolgu yoktur. Çıktı = $((27 + 2*0 - 3)/2) + 1 = 13$ olur. Çıkış özellik haritası da 13x13x256 olur.

5. Katman: Her biri 3x3 boyutunda olan 384 filtreye sahip bir konvolüsyonel katmanıdır. Stride (adım sayısı) 1'dir. Dolgu 1'e eşittir. ReLU aktivasyonu kullanılır. Çıktı = $((13 + 2*1 - 3)/1) + 1 = 13$ olur. Çıkış özellik haritası da 13x13x384 olur.

6. Katman: Her biri 3x3 boyutunda olan 384 filtreye sahip bir konvolüsyonel katmanıdır ve bir önceki konvolüsyonel katmanındaki filtrelerin yalnızca belirli bir alt kümesine bağlanır. Stride (adım sayısı) 1'dir. Dolgu 1'e eşittir. ReLU aktivasyonu kullanılır. Çıktı = $((13 + 2*1 - 3)/1) + 1 = 13$ olur. Çıkış özellik haritası da 13x13x384 olur.

7. Katman: Her biri 3x3 boyutunda olan 256 filtreye sahip bir konvolüsyonel katmanıdır ve burada da bağlantılar sınırlıdır. Stride (adım sayısı) 1'dir. Dolgu 1'e eşittir. ReLU aktivasyonu kullanılır. Çıktı = $((13 + 2*1 - 3)/1) + 1 = 13$ olur. Çıkış özellik haritası da 13x13x256 olur.

8. Katman: 3x3 filtre boyutunda, 2 adımlı maksimum havuzlama katmanı uygulanır. Dolgu yoktur. Çıktı = $((13 + 2*0 - 3)/2) + 1 = 6$ olur. Çıkış özellik haritası da 6x6x256 olur.

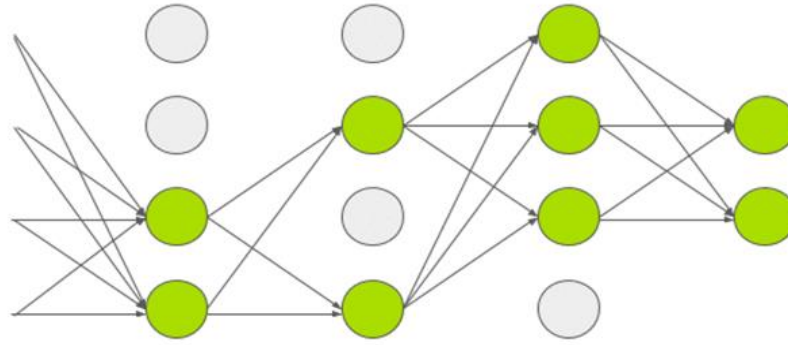
9. Katman: 4096 nörona sahip tam bağlantılı bir sinir ağıdır. 9216(6x6x256) boyutunda bir girdi geçer. ReLU aktivasyonu kullanılır.

10. Katman: 4096 nörona sahip tam bağlantılı bir sinir ağıdır. ReLU aktivasyon fonksiyonu kullanılır.

11. Katman: 1000 nörona sahip tam bağlantılı bir sinir ağıdır ve aynı zamanda çıktı veren katmandır. Softmax aktivasyon fonksiyonu kullanılır [25], [26], [27].

Maksimum havuzlama, özellik haritasındaki her penceredeki maksimum değeri alarak önemli özellikleri korurken aynı zamanda boyutunu küçültmek amacıyla yapılan bir işlemdir. AlexNet'te bu adım 2 kere uygulanır ki bunun sebebi hem yatay boyutta hem dikey boyutta veri yarıya indirilsin bunun sonucunda daha az efor ve bellek kullanımıyla bir çıkış haritası elde edilsin [25].

Dropout, eğitim sırasında kullanılan aşırı öğrenmeyi önlemek amacıyla kullanılan bir tekniktir. AlexNet'te fully connected katmanlarında kullanılır ve modelin genelleme yeteneğini artırarak özelleşmesini engeller. Rastgele seçilmiş bir kısım nöronların ağırlıklarını sıfırlayarak öğrenmesini engellemiş olur. AlexNet mimarisinde bu dropout oranı 0.5 olarak uygulanmıştır. Yani her bir nöronun devre dışı bırakılma oranı %50 olmuş olur [25].



Şekil 3.4.2. Dropout mantığı [28]

AlexNet, sekiz katmanlı bir derin ağ modeli kullanarak kendisinden önceki modellere göre çok yüksek bir performans göstermiş ve VGG, ResNet gibi modelleri büyük ölçüde etkilemiştir. Yenilikçi teknikler kullanmış olması derin öğrenmedeki birçok probleme çözüm bulmuştur.

Zamanla yeni teknolojiler geliştikçe, modern derin öğrenme yöntemlerine kıyasla AlexNet bazı dezavantajlara sahip olmuştur. Büyük bir ağ yapısına sahip olduğu için yüksek hesaplama gücü gerektirir ve dolayısıyla eğitim süresi de uzun sürer. Overfitting önlemek amacıyla dropout tekniği kullanılsa da yine de aşırı uyumlanmaya yol açabilecek bir ağ yapısına sahipti. Aynı zamanda AlexNet, ImageNet verisetine göre eğitildiği için daha karmaşık ve daha küçük veri setlerinde aynı performansı göstermiyordu.

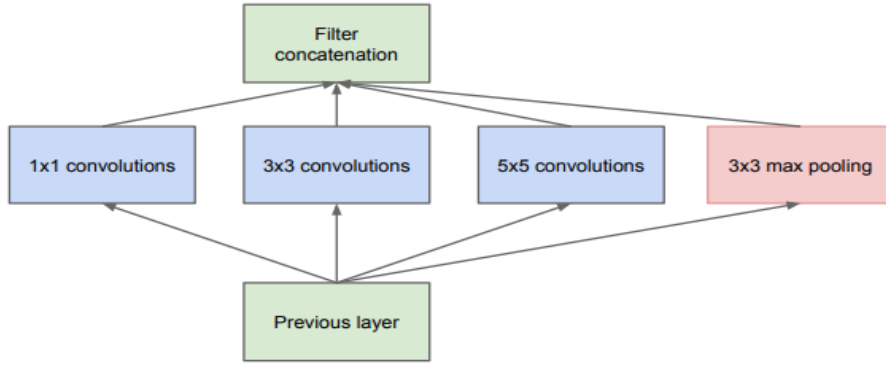
INCEPTIONNET

İlk olarak ILSVRC 2014 yarışmasında "Going Deeper with Convolutions" başlıklı makalede önerilen InceptionNet, derin öğrenme alanındaki sınıflandırma ve tespit zorluklarına bir çözüm olarak sunulmuş ve deneysel çalışmalar sonucunda da önemli bir ölçüde daha iyi performans gösterdiği doğrulanmıştır [28].

Modelin hem derinliğini hem de genişliğini artırmak (yani modelin boyutunu artırmak), ağın performansını da artıracaktı. Ancak bu çözümün iki önemli dezavantajı da olacaktı. Bunlardan biri modelin boyutunu artırmak için daha fazla sayıda parametre gerektirmesi ki bu da aşırı uyuma yol açabilir, diğeri de artan ağ boyutu sonucu hesaplama maliyetinin de aynı şekilde artacağıydı çünkü uygulamada hesaplama bütçesi her zaman sınırlıdır.

Inception mimarisinin, görüntü ağırları için daha verimli bilgi işlemeye olanak tanıyan seyrek bir yapıyı tahmin etmeyi amaçlayarak çalışmalarına başlanmıştır. Modelin üzerinde birkaç iyileşmelerin yapılması sonucunda özellikle yerleştirme ve nesne tespitinde başarılı olduğu görülmüştür. Ancak bu mimari, bilgisayarlı görü için başarılı olsa da emin olmak için daha kapsamlı bir analiz ve doğrulama gerektirir.

Inception mimarisi, farklı boyutlardaki filtreleri paralel olarak ekler ve bunların sonuçlarını birleştirir. Bu yapıya da "Inception Module" adı verilir.

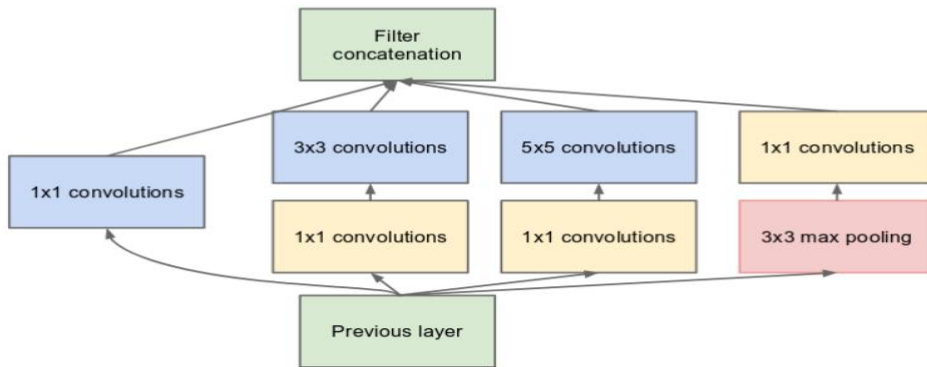


(a) Inception module, naïve version

Şekil 3.5.1. InceptionNet mimarisi [28]

Derin sinir ağıları, hesaplama açısından maliyetli olduğu için bu maliyeti düşürmek için 3x3 ve 5x5 evrişim filtrelerinden önce fazladan bir 1x1 evrişim filtresi ekleyerek giriş kanallarının sayısını sınırlandırmışlardır. Bunun yanında ise doğrusal olmayan aktivasyonlar ekleyerek modelin öğrenme kapasitesini artırarak 1x1 evrişimlerini çift amaçlı kılmıştır [28].

Maksimum havuzlamada ise şu fark vardır, 1x1 evrişim filtresi önce değil sonra eklenir ve uygulanır.



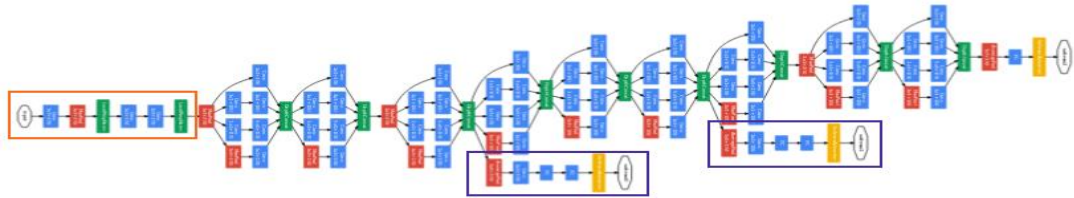
(b) Inception module with dimension reductions

Şekil 3.5.2. Hafifletilmiş InceptionNet mimarisi [28]

Inception mimarisinin temeli, giriş verisinin farklı ölçeklerde çeşitli özellik haritalarını öğrenmek ve daha sonra bunları bir araya getirerek bu giriş verilerinin daha kapsamlı bir halini oluşturabilmektir. Bu yöntem, görüntü sınıflandırma gibi birçok derin öğrenme alanında yararlı olabilecektir. Bundan dolayı da zaten tıbbi alanda da bu yöntem kullanılmaktadır.

Inception mimarisi, öncelikle giriş verilerine paralel olarak farklı boyutlardaki evrişimli filtrelerini uygular. Böylelikle bu farklı boyuttaki filtreler (3x3, 5x5 veya 1x1), farklı boyuttaki özellikleri öğrenmiş olurlar. Bu giriş verilerinin örtüşmeyen bölgeleri için maksimum havuzlama katmanı uygulanır. Ardından ise her filtrenin çıktısı tek bir çıktı özellik haritası oluşturulmak üzere birleştirilir [28].

Modeli daha fazla derinleştirmek istendikçe modele daha fazla Inception modülü eklenir. Bundan dolayı birden fazla sürüme sahiptir. Yaygın adıyla GoogLeNet olarak bilinen mimari, Inception ağı mimarisinin ilk sürümü olan Inception v1'dir. 22 katman derinliğinden oluşan GoogLeNet, farklı ölçeklerde maksimum havuzlama katmanları ile toplam 9 Inception modülü içerir [28].



Şekil 3.5.3. GoogLeNet mimarisi [28]

Inception v2; her evrişimsel katmandan sonra bir toplu normalizasyon katmanı kullanmıştır. Ayrıca 5x5 konvolüsyonu yerine iki ardışık 3x3 konvolüsyonu kullanarak maliyeti düşürmüştü ve modelin performansını artırmıştır [29].

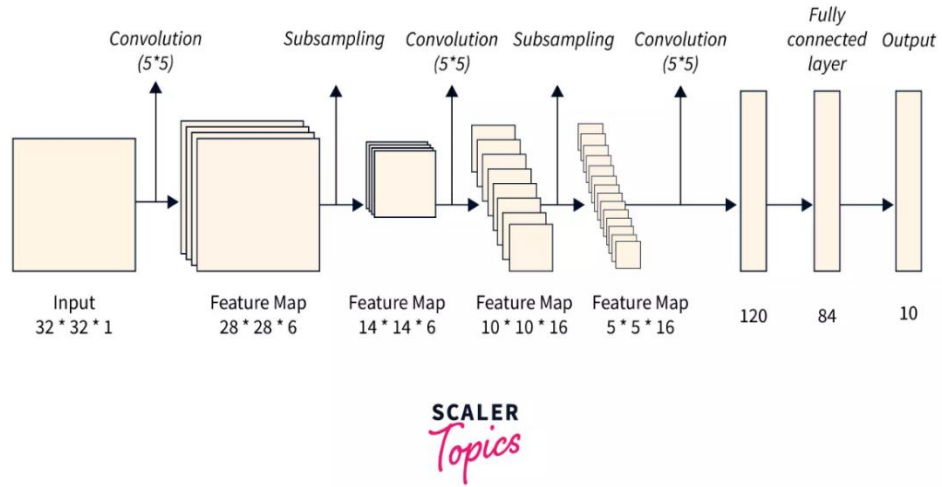
Inception v3; hem toplu normalizasyon katmanı kullanarak hem de 3x3 konvolüsyonları yerine 1x3 ve 3x1 şeklinde iki ayrı filtre uygulayarak modeli derinleştirmiş ve doğruluk oranını yükseltmiştir [29].

Inception v4; diğer üç sürümden farklı olarak, bir araya getirilmiş birden fazla Inception modülünden oluşan bir ağ mimarisi tasarlanıp kullanılmıştır ve daha büyük veri setlerinde daha iyi sonuç vermiştir [29].

Inception-ResNet; son sürüm olarak Inception modüllerine residual bloklar eklemişlerdir ve bu bloklar sayesinde gradyan kaybolma sorunu ile modelin daha etkili öğrenmesi üzerinde iyileştirme sağlamışlardır [29].

LENET

90lı yıllara dayanan LeNet mimarisi, ilk olarak el yazısı ve makinede basılmış karakterleri tanımak için geliştirildi ve karakterlerin yazılı olduğu 28x28 boyutunda resimler bulunan MNIST gibi veritabanlarında başarılı sonuç elde etmiştir. Giriş ve çıkış katmanları hariç 7 katmandan oluşur, bunlar da üç evrişimsel katman, iki alt örnekleme katmanı ve iki tam bağlı katmandır [30], [31].



Şekil 3.6.1. LeNet mimarisi [32]

Bu modelin girdisi 32x32 gri tonlamalı bir görüntü olduğundan kanal sayısı 1'dir.

C1 - Evrişim Katmanı: İlk evrişim işlemi burada gerçekleşir. Girdiye 5x5 filtre boyutuyla 6 filtre (çekirdek) uyguluyoruz ve sonuç olarak 28x28 boyutunda 6 adet özellik haritası elde ederiz. Burada kanal sayısı uygulanan filtre sayısına eşittir.

Ayrıca convolution işlemi sırasında valid padding (kenarlara hiçbir işlem yapılmayarak giriş boyutunu küçültmek) uygulandığında, çıktı boyutu şu formülle hesaplanır: Çıkış boyutu = Girdi boyutu – Kernel boyutu + 1

S2 - Alt Örnekleme Katmanı: Burada 2x2 filtre boyutuyla 2'lik bir adım kullanarak ortalama havuzlama işlemi gerçekleşir ve sonuç olarak 14x14 boyutuna sahip 6 adet özellik haritası elde etmiş oluruz. Özellik haritasının boyutu yarı yarıya azalmış olup, kanal sayısı aynı kalmıştır.

C3 - Evrişim Katmanı: Daha karmaşık özellikleri öğrenmek için daha fazla filtre içeren bu katmanda 5x5 filtre boyutuyla 16 filtre uygulanır. Çıkış haritası olarak ise 10x10x16 elde etmiş oluruz.

S4 - Alt Örnekleme Katmanı: Bu katmanda 2x2 filtre boyutuyla 2'lik bir adım kullanarak ortalama havuzlama işlemi gerçekleşir ve sonuç olarak 5x5 boyutuna sahip 16 adet özellik haritası elde etmiş oluruz. Özellik haritasının boyutu yarı yarıya azalmış olup, kanal sayısı aynı kalmıştır.

C5 - Tam Bağlantılı Katman/ Evrişim Katmanı: Bu katman; her biri 1x1 ölçülerinde olan 120 özellik haritasındaki her birim, dördüncü katmandaki (S4) tüm düğümlere (5x5x16) bağlı olan tam bağlı bir convolution katmanıdır. Özellik haritası boyutu 1x1x120 olmuş oldu.

F6 - Tam Bağlantılı Katman: Bu katman, 84 nöronlu tam bağlı bir katmandır ve aktivasyon fonksiyonu olarak tanh kullanılır.

Çıktı katmanında ise aktivasyon fonksiyonu olarak Softmax kullanıldığından ve 0-9 rakamlarına karşılık gelen 10 potansiyel değere sahip olduğundan on nöronlu çıktı katmanı oluşmuş olur [30], [31], [32].

Öğrenilebilir parametrelere sahip 5 katman bulunan ve eğitilebilen parametre sayısı 60 bin civarı olan LeNet mimarisi, basit olmasından kaynaklı düşük hesaplama maliyeti ile kenar algılamada derin öğrenme alanındaki birçok konuda öncü olmuştur.

Layer		Feature Map	Size	Kernel Size	Stride	Activation
Input	Image	1	32x32	-	-	-
1	Convolution	6	28x28	5x5	1	tanh
2	Average Pooling	6	14x14	2x2	2	tanh
3	Convolution	16	10x10	5x5	1	tanh
4	Average Pooling	16	5x5	2x2	2	tanh
5	Convolution	120	1x1	5x5	1	tanh
6	FC	-	84	-	-	tanh
Output	FC	-	10	-	-	softmax

Şekil 3.6.2. LeNet mimarisinin geniş gösterimi [30]

EFFICIENTNET

Model geliştirmek çoğunlukla sınırlı bir bütçe ile gerçekleştirildiği için; ağ derinliğini, ağ genişliğini ve ağ çözünürlüğünü bir bileşik katsayı kullanarak dengeleyerek EfficientNet modeli ortaya çıkmıştır.

2019 yılında tanıtılan bu mimari, tıbbi alan ve daha birçok alanda da görüntü sınıflandırma işlemlerinde, obje tespiti işlemlerinde ve video analizi gibi görüntü görevlerinde kullanılmaktadır. Sekiz farklı sürümüne sahiptir (EfficientNet-B0,B1, ..., B7). EfficientNet-B0 en temel modelidir ve MobileNetV2 esas alınarak geliştirilmiştir [33].

ConvNet'lerin ölçeğini büyütmek ne kadar yaygın olarak kullanılsa da üç boyuttan (derinlik, genişlik ve görüntü boyutu) yalnızca biri ölçeklendiriliyordu. Bundan da dolayı bu teknik, EfficientNet'te diğerlerine kıyasla çok daha iyi hız ve performans göstermektedir [34].

Genişliği, derinliği veya boyutu rastgele ölçeklendirme yerine, dengeli ölçeklendirme için her biri belirli bir sabit katsayı ile ayarlanır. Örneğin, 2^N kat daha fazla hesaplama kaynağı kullanmak istiyorsak; ağ derinliğini α^N , genişliğini β^N ve görüntü boyutunu γ^N kadar artırabiliriz. Burada α , β , γ belirlenen sabit katsayılardır ve görülmüştür ki düzenli bir evrişimin FLOPS'u d , w^2 , r^2 ile orantılıydı. Mesela ağ derinliğini k katına

çıkartılması FLOPS'un k katına çıkmasına neden olur, ancak çözünürlüğün k katına çıkartılması FLOPS'u dört kat artırır [34].

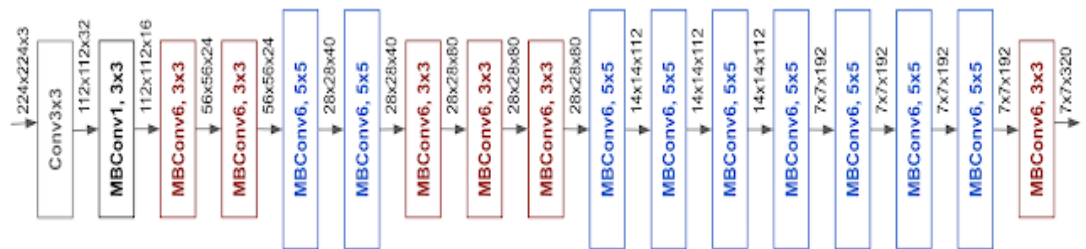
Sahne	Operatör	Çözünürlük	#Kanallar	#Katmanlar
1	Conv3x3	224 × 224	32	1
2	MBConv1, k3x3	112 × 112	16	1
3	MBConv6, k3x3	112 × 112	24	2
4	MBConv6, k5x5	56 × 56	40	2
5	MBConv6, k3x3	28 × 28	80	3
6	MBConv6, k5x5	14 × 14	112	3
7	MBConv6, k5x5	14 × 14	192	4
8	MBConv6, k3x3	7 × 7	320	1
9	Conv1x1 & Havuzlama & FC	7 × 7	1280	1

Şekil 3.7.1. EfficientNet mimarisi adımları [34]

EfficientNet-B0 mimarisi, en temelde 3 kısımdan oluşuyor. İlk kısım; giriş boyutunu 224x224 çözünürlükte alan, 32 filtreden oluşan, 2 adımlı bir konvolüsyon katmanıdır ve hemen ardından toplu normalizasyon ile Swish aktivasyon fonksiyonu uygulanır.

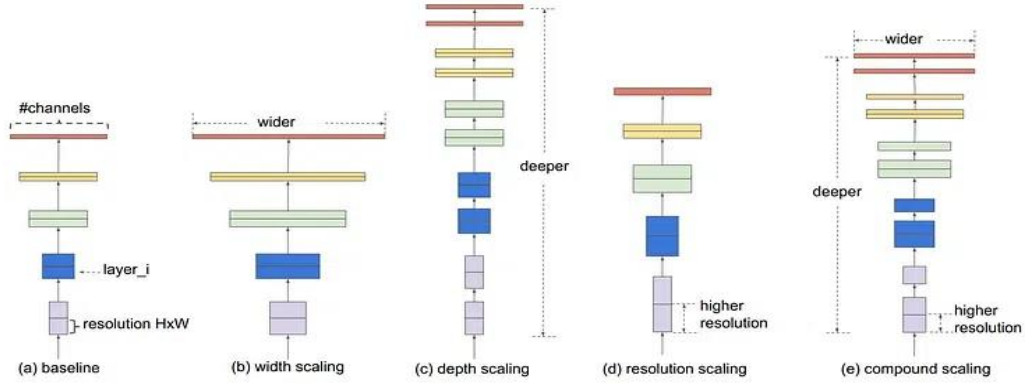
İkinci kısım; farklı yapılar sahip bottleneck yapısıyla çalışan bir kısım MBConv bloğundan oluşur. Burada daha fazla bilgi öğrenebilmek için veri boyutu geçici olarak genişletilir, depthwise konvolüsyon işleminden geçirilir ardından özellik haritaları orijinal boyutuna geri döndürülür ve eğer giriş ile çıkış boyutları aynıysa bir “atlama bağlantısı” eklenir. Her MBConv bloğu SE (squeeze-and-excitation) katmanı içerir. Bu katman her kanala ağırlık çarpanı uygulayarak önemli kanalların önemini artırırken önemsiz kanalları zayıflatır.

Üçüncü yani son kısım; sınıflandırma için softmax aktivasyon fonksiyonuna sahip tam bağlantılı katmandır ve burada son konvolüsyon katmanından ardından küresel ortalama havuzlama yapılır [33], [34].



Şekil 3.7.2. EfficientNet mimarisi [35]

Compound Scaling, ölçeği üç boyutta da (genişlik, derinlik ve görüntü çözünürlüğü) dengeleme işlemidir. Modelin performansı tek bir boyutu ölçeklendirme ile de iyileşirken 3 boyutta da ölçeklendirmek en iyi performansa ulaştırmışlardır. Şekil farklı ölçeklendirme yöntemlerini göstermektedir.



Şekil 3.7.3. *EfficientNet* mimarisi farklı ölçeklendirme yöntemleri gösterimi [34]

Düşük maliyetli cihazlarda bile ve az bir parametreyle ve FLOP ile çalışan EfficientNet, Compound Scaling yöntemi ile yüksek başarı oranı yakalamaktadır. Öyle ki ImageNet'te %84,3 doğruluğa ulaşırken ResNet 152'den daha iyi hız performansı sergilemektedir [34].

MOBILENET

MobilNet mimarisi, ilk olarak 2017 yılında "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications" adlı makalede yayınlanmış olup mobil ve gömülü görüntü uygulamaları için basit ama etkili bir yol olmuş ve ImageNet sınıflandırmasında diğer mimarilerle de kıyasla yüksek başarı oranı sağlamıştır. Nesne-yüz algılama ile görüntü sınıflandırmada yaygın olarak kullanıldığından sağlık, eğlence, trafik gibi birçok gerçek dünya alanında ve küçük boyuta sahip olmalarından dolayı günümüzdeki mobil cihazlarda da etkisini göstermektedir. Çok küçük ve verimli dolayısıyla hızlı bir mimaridir [36].

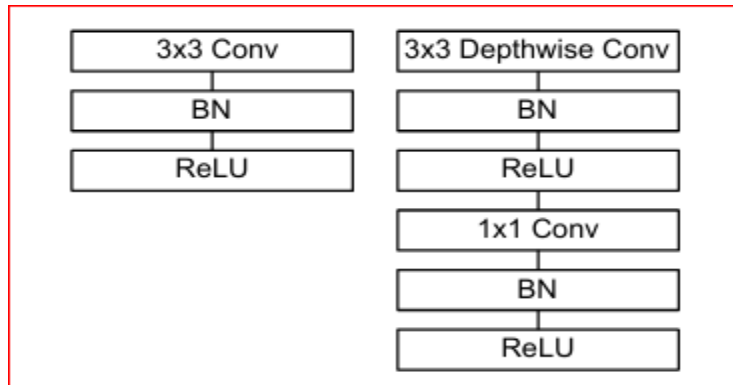
Gecikmeyi optimize eden MobilNet'te, gecikme ve doğruluk arasında denge kuran iki hiper parametre (width multiplier ve resolution multiplier) kullanılır. Modelin genişliği ile özellik haritalarının ve giriş görüntüsünün çözünürlüğünü ayarlayan bu hiper

parametreler, kısıtlamalar dahilindeki doğru boyutlu modeli oluşturmada görev almış olur [36], [37].

Standart evrişimli katmanlar yerine derinlemesine ayrılabilir evrişimli katmanlar kullanılır ve bu ikisinin arasındaki en temel fark derinlemesine ayrılabilir evrişimli katmanların her giriş kanalına tek bir filtre uygulaması ve 1x1 evrişim filtresi kullanarak farklı kanalları birleştirip parametre sayısını azaltmasıdır. Bu, filtrelerin tüm giriş kanallarına uygulandığı standart evrişimden farklı olmasıyla birlikte daha hafif, daha hesaplı ve daha hızlı sonuç vermiş olur. Yani verimliliği çok yüksektir [36].

Derinlemesine ayrılabilir evrişim (Depthwise Separable Convolution),”derinlik bazlı evrişim ve nokta bazlı evrişim olmak üzere iki kısımdan oluşur.”İlk katmanda her bir giriş kanalına ayrı ayrı filtre uygulanıp çıkış özellik haritaları elde edilir. İkinci katmanda ilk katmandan elde edilen çıkış özellik haritalarına 1x1 filtresi uygulayarak farklı kanalları birleştirerek kombinasyon oluşturur [36], [37].

Bu şekilde iki ayrı işlem olarak evrişimin gerçekleşmesi, gereken parametre ve hesaplama sayısını azaltırken, modelin derinliğini ve doğruluğunu olumlu yönde etkiler. Bu da MobileNet mimarisini özellikle sınırlı hesaplama kaynaklarına sahip olan mobil ve gömülü cihazlar için kullanılabilir hale getirmiş olur.



Şekil 3.8.1. Standart Evrişim Katmanı vs. Derinlemesine Ayrılabilir Evrişim Katmanı [36]

Derinlik ve noktasal evrişimi de ayrı olarak sayıldığında 28 katmana sahip MobileNet’in ilk katmanı tam evrişim katmanı, diğer katmanları ise derinlemesine ayrılabilir evrişim katmanlarıdır. Tam evrişim katmanı olan ilk katman, 224x224x3 RGB görüntüye standart bir 3x3 evrişim işlemi uygular. Ardından derinlemesine

ayrılabilir evrişim katmanlarında; giriş özellik haritalarına, her giriş kanalının ayrı ayrı filtre uygulanması ile çıkış özellik haritaları elde edilir. Bu çıkış özellik haritalarına 1x1 filtresi ile evrişim uygulanır. Yani filtreleme ve birleştirme işlemleri uygulanmış olur. Ayrıca tam bağlı katman haricinde tüm katmanlarda toplu normalizasyon ve ReLU aktivasyon fonksiyonu adımları uygulanır. Ardından havuzlama işlemi yapılır ve tam bağlantılı katmanda sınıflandırma yapıp softmax aktivasyonu ile sonuçlandırılır [36].

Table 1. MobileNet Body Architecture

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5x	Conv dw / s1	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 512$
	Conv dw / s2	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 1024$
	Conv dw / s2	$3 \times 3 \times 1024$ dw
	Conv / s1	$1 \times 1 \times 1024 \times 1024$
	Avg Pool / s1	Pool 7×7
	FC / s1	1024×1000
	Softmax / s1	Classifier

Şekil 3.8.2. MobileNet mimarisi adımları [36]

MobileNet, küçük ve çok verimli derin öğrenme mimarisidir. Bu verimliliğinden kaynaklı diğer evrişim mimarileriyle kıyasla çok daha az parametre gerekliliği ile çok yakın doğruluk oranları yakalamaktadır. Örneğin Stanford köpekleri verisetinde, Inception V3 modeli 23,2 milyon parametresi ile %84 doğruluk elde ederken, MobileNet 3,3 milyon parametresi ile %83,3 doğruluk elde eder. MobileNet; VGG16'ya göre 32 kat daha küçük ve 10 kat daha hızlıdır, GoogleNet'e göre daha doğru ve daha küçüktür aynı zamanda 2,5 kattan daha az hesaplama gerektirir, AlexNet'ten 45 kat daha küçüktür, yaklaşık aynı boyutta ve 22 kat daha az hesaplama ile Squeezenet'ten %4 daha iyidir [36].

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
0.50 MobileNet-160	60.2%	76	1.32
Squeezenet	57.5%	1700	1.25
AlexNet	57.2%	720	60

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
GoogLeNet	69.8%	1550	6.8
VGG 16	71.5%	15300	138

Şekil 3.8.3. MobileNet mimarisinin diğer popüler mimariler ile karşılaştırma sonuçları [36]

3 farklı versiyona sahiptir. MobilNetV1; ilk mimaridir, en hızlı ve en küçük olandır, diğerlerine kıyasla daha düşük doğruluk oranına sahiptir. MobileNetV2; birkaç farklı teknik (bottleneck layers) kullanarak daha iyi doğruluk oranı yakalamıştır. MobileNetV3; hardswish aktivasyon fonksiyonu kullanmış olup en iyi doğruluk oranını yakalamıştır [38].

MALZEME VE YÖNTEM

Covid-19 tespiti için 15.000 civarı görüntü kullanılarak iki popüler derin öğrenme önceden eğitilmiş CNN modeli kullanılmış ve karşılaştırılmıştır. Bunlar ResNet101 ve DenseNet201 modelleridir. ResNet, derin öğrenme alanında katmanlar arasında doğrudan bağlantılar kurarak gradyan sönmesi problemini ortadan kaldıran bir yapıya sahiptir. DenseNet ise geleneksel bir CNN modelinden daha az parametre gerektirir çünkü bu, yedekli özellik haritaları üzerinde eğitim gerektirmez. DenseNet mimarisinin çok dar katmanları vardır, bu nedenle yalnızca az sayıda yeni özellik haritası ekler.

Veri seti eğitim, doğrulama ve test olmak üzere üç alt kümeye ayrılmıştır. Rastgele ayırım sonucunda veri setinin %70'i eğitim, %15'i doğrulama ve %15'i test amaçlı olarak kullanılmıştır. Bu oranlama, modelin eğitim sürecinde öğrenme başarısını artırırken, doğrulama ve test süreçlerinde overfitting gibi sorunları gözlemleyebilmek adına tercih edilmiştir.

Tüm görüntüler 224×224 piksel çözünürlüğe ölçeklendirilmiş, RGB kanalları kullanılarak 3 boyutlu hale getirilmiştir. Çeşitli şekillerde kaydırma, kesme, dönüşüm, zoom ve ışıpta oynama işlemleri yapılmış olup piksel yoğunluk değerleri 0–255 aralığından [0, 1] aralığına normalize edilerek modelin daha hızlı ve sağlıklı öğrenmesi sağlanmıştır. Sınıf etiketleri, model tarafından anlaşılabilir hale getirilmesi için one-hot encoding yöntemiyle sayısal değerlere dönüştürülmüştür.

GPU destekli Google Colab ortamında eğitim gerçekleştirilmiştir. Eğitim boyunca TensorFlow ve Keras kütüphaneleri kullanılmış olup Python dili tercih edilmiştir. Her iki model de 10 epoch eğitilmiş ve öğrenme oranı başlangıçta 0.001 olarak belirlenmiştir. Optimize edici olarak Adam algoritması tercih edilmiştir ve kayıp fonksiyonu olarak çok sınıflı sınıflandırmalarda yaygın kullanılan categorical crossentropy kullanılmıştır. Modelin aşırı öğrenmesini engellemek amacıyla EarlyStopping stratejisi uygulanarak doğrulama kaybı belirli bir süre iyileşmediğinde eğitim süreci sonlandırılmıştır. Ayrıca ReduceLROnPlateau yöntemi ile doğrulama kaybının sabit kalması durumunda öğrenme oranı otomatik olarak azaltılmıştır.

Modelin başarısını deęerlendirmek iin doęruluk (accuracy) ve kayıp (loss) deęerleri takip edilmiřtir. Bunun yanında, confusion matrix (karmařıklık matrisi), precision (kesinlik), recall (duyarlılık) ve F1-score gibi ek deęerler modelin sınıflar arası ayırt edicilięini lmek iin deęerlendirilmiřtir. Tm bu deęerler modelin hem eęitim hem de doęrulama verisi zerindeki başarısını incelemek iin grafięe dklmřtr.

ResNet mimarisini oluřtururken ncelikle bir residual blok tanımladım. Bu blok iki convolsyon katmanı ve bu katmanlar arasında BatchNormalization ve ReLU aktivasyonu ierir. Ayrıca giriřin (shortcut) doęrudan ıkıřa eklendięi bir artık baęlantı da vardır. Daha sonrasında ise ana mimarimi oluřturdum. İlk katmanlarım giriř grntsne konvolsyon, BatchNormalization, ReLU ve MaxPooling uygulamaktadır. Ardından sırayla 3 adet 64 filtreli blok, 4 adet 128 filtreli blok, 6 adet 256 filtreli blok, 3 adet 512 filtreli ResNet blokları yer almaktadır. Bu toplamda 101'e yakın katman anlamına gelmektedir. Son olarak mimarinin sınıflandırma katmanlarına geldik. GlobalAveragePooling2D ile parametre sayısını azaltıp, dropout ile %50 oranında nron kapatarak overfitting'i nlemeye alıřtı ve softmax ile olasılık ıktısını verdi.

DenseNet mimarisini oluřtururken ise ImageNet veri kmesi zerinde nceden eęitilmiř aęırlıkları kullandım. Modelin nceki ęrendiklerini korumasını saęladım ki bu sayede daha az hesaplama yapılsın ve modelin ezberlemesi engellenmiř olsun. Ardından sınıflandırma katmanlarını ekledim. GlobalAveragePooling2D ile parametre sayısını azaltıp overfitting oluřumunu engellemeye alıřıldı. Bunun yanında extra olarak dropout kullanıldı. Softmax ile 3 sınıflı olasılık ıktısı retildi. Model tanımlandı ve derlendi. Derlerken hızlı ve iyi sonu veren optimizasyon algoritması olan adam kullanıldı. Eęitim sresini kısaltmak ve overfitting nlemek iin callbacks fonksiyonu ierisinde EarlyStopping, restore_best_weights ve ReduceLROnPlateau tanımladım. EarlyStopping, doęrulama kaybı 5 epoch boyunca iyileřmezse eęitimi durduracak. ReduceLROnPlateau, doęrulama performansı 3 epoch boyunca sabitse, ęrenme oranını 0.1 katına dřrecek.

BULGULAR VE TARTIŞMA

Bu çalışmada, Covid-19, Normal ve Viral Pnömoni sınıflarının akciğer röntgen görüntülerinden doğru bir şekilde sınıflandırılmasını sağlamak amacıyla iki farklı derin öğrenme mimarisi olan ResNet101 ve DenseNet201 karşılaştırmalı olarak eğitilmiş ve değerlendirilmiştir. Her iki model de transfer öğrenme yöntemleri ile önceden ImageNet veri kümesi üzerinde eğitilmiş ağırlıklarla başlatılmış, üst katmanları medikal görüntü sınıflandırmasına uygun biçimde yeniden yapılandırılmıştır. Bu bölümde, her iki modelin eğitim ve doğrulama verileri üzerindeki başarıları analiz edilmekte; doğruluk (accuracy), kayıp (loss), sınıf bazlı metrikler ve hata örüntüleri karşılaştırmalı olarak değerlendirme yapılmıştır.

ResNet101 mimarisi, 10 epoch süren eğitim boyunca eğitim doğruluğunu %75'ten %89'a çıkarmış; eğitim kaybını 0.58'den 0.28'e düşürmüştür. Doğrulama doğruluğu ilk epoch'ta %89.5 olarak gözlemlenmiş, 4. epoch sonrası %92 civarında sabitlenmiştir. DenseNet201 modeli ise eğitimin başından itibaren hem eğitim hem de doğrulama süreçlerinde daha stabil bir performans sergilemiştir. Eğitim doğruluğu benzer şekilde %75 seviyelerinden başlayıp %89'a ulaşırken, doğrulama doğruluğu %93'e kadar yükselmiş ve kayıp değerlerinde daha düzenli bir düşüş görülmüştür.

Her iki modelde de EarlyStopping (erken durdurma) ve ReduceLROnPlateau (öğrenme oranı azaltma) mekanizmaları kullanılmış, eğitim sürecinde zaman zaman overfitting gözlenmiş olsa da bu büyük oranda azaltılmaya çalışılmıştır. Aynı zamanda grafik eğrileri karşılaştırıldığında, ResNet101 modelinin doğrulama doğruluğunda dalgalanmalar yaşadığı; DenseNet201'in ise daha stabil bir doğrulama eğrisi sunduğu dikkat çekmiştir. Bu durum, DenseNet'in daha iyi genelleme yeteneğine sahip olduğunu göstermektedir.

Her iki model için oluşturulan confusion matrix ve classification report çıktıları, sınıflar arası başarıları detaylandırmak amacıyla değerlendirilmiştir. Her iki model de Covid-19 sınıfını yüksek başarıyla sınıflandırmıştır. Özellikle DenseNet201 modelinde Covid-19 sınıfı için precision ve recall değerleri %95'in üzerine çıkmıştır. ResNet101 modelinde ise bu oran %90–%92 aralığında seyretmiştir.

Normal sınıf, her iki model tarafından benzer başarı oranlarıyla tanınmıştır. Fakat en dikkat çekici fark Viral Pnömoni sınıfında ortaya çıkmıştır. ResNet101, Viral Pnömoni sınıfında daha fazla hata yaparken; bu sınıfı zaman zaman COVID-19 ile karıştırmıştır. DenseNet201 ise bu sınıfı daha doğru sınıflandırarak daha dengeli precision ve recall değerleri sunmuştur. Özellikle F1-score değerleri karşılaştırıldığında, DenseNet201 tüm sınıflarda daha yüksek ortalamalara ulaşmıştır.

Model mimarileri göz önüne alındığında, ResNet101, derinliği sayesinde yüksek kapasiteli bir öğrenme imkânı sunarken; doğrulama sürecinde bazı dalgalanmalar göstermiştir. Bunun temel nedeni, ResNet'in her katmanı arasında yalnızca bir önceki çıktıyı alan geleneksel “artık bağlantılar” ile çalışmasıdır. Bu bağlantılar, çok derin yapılarda gradyan kaybını azaltmak için faydalı olsa da, zaman zaman genelleme problemleriyle karşılaşılabilir.

Buna karşılık DenseNet201 mimarisi, her katmandan gelen çıktıyı sonraki tüm katmanlara aktaran yoğun bağlantı mekanizması sayesinde hem gradyan yayılımını artırmış hem de bilgi kaybını azaltmıştır. Bu özellik, özellikle sınıflar arası ayrımın belirsiz olduğu medikal görüntülerde kritik önem taşımaktadır. DenseNet'in daha stabil bir öğrenme eğrisi göstermesinin nedeni bu mimari avantaj olabilir.

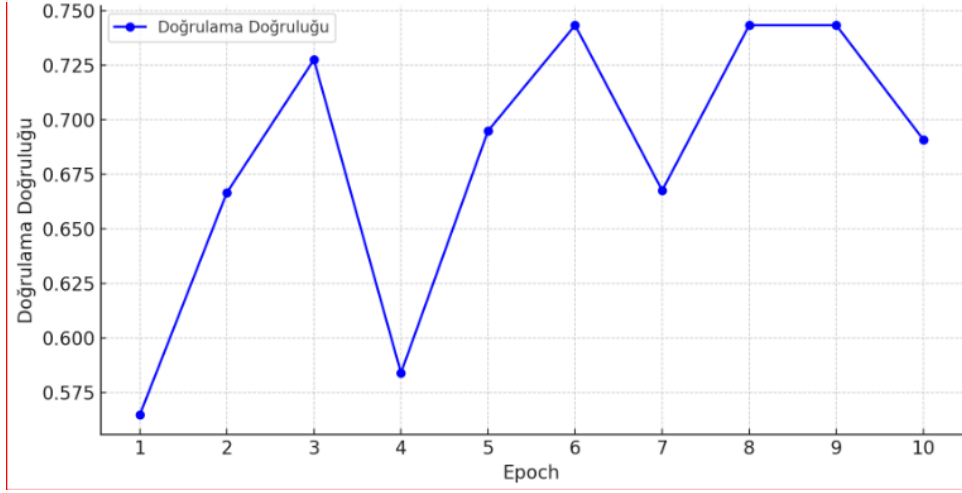
Her iki modelin eğitim süresi ve kaynak kullanımı da değerlendirilmiştir. ResNet101 modeli, daha derin yapıda olduğu için her epoch başına daha fazla hesaplama süresi gerektirmiştir. DenseNet201 modeli ise bağlantı yoğunluğuna rağmen daha hızlı konverge olmuş ve eğitim süresi daha kısa sürmüştür. Bu durum, pratik uygulamalarda DenseNet'in daha avantajlı olabileceğini düşündürmektedir.

Hata örüntüleri incelendiğinde, her iki model de Viral Pnömoni sınıfında hata yapmaya daha yatkın olmuştur. Bu hata tipi genellikle COVID-19 ile karışıklık şeklinde gerçekleşmiştir. Görsel benzerlikler, sınıflar arasında dokusal ve yapısal benzerlikler nedeniyle, sınıflandırma başarısını düşürmektedir. Bu da veri setinin sınıf içi varyansını azaltmak ya da daha güçlü özellik çıkarıcılar (örneğin segmentasyon ön işlemleri) kullanmanın potansiyel faydalarına işaret etmektedir.

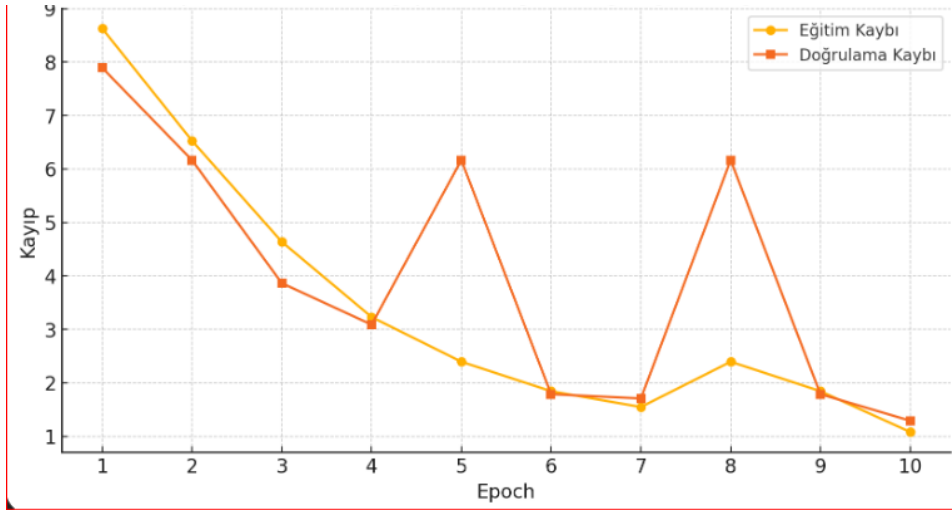
Sonuç olarak, her iki model de COVID-19 sınıflandırması için yüksek doğruluk oranlarıyla çalışmakta; ancak DenseNet201 modeli hem daha stabil bir öğrenme eğrisi sunmakta hem de sınıflar arası genel başarıda öne çıkmaktadır. ResNet101 mimarisi güçlü bir öğrenme yapısına sahip olsa da doğrulama verisinde zaman zaman dalgalı bir performans sergilemiş, özellikle Viral Pnömoni sınıfında daha düşük başarı göstermiştir. DenseNet201, daha az hata yaparak ve daha az epoch'ta daha yüksek doğruluklara ulaşarak bu alanda daha başarılı bir çözüm sunmuştur.

Bu bulgular, medikal görüntü sınıflandırmasında yalnızca model derinliğinin değil, bağlantı yapısının da hayati öneme sahip olduğunu ortaya koymaktadır. Gelecek çalışmalarda, DenseNet gibi yoğun bağlantılı yapılarla birlikte dikkat mekanizmaları, segmentasyon ön işlemleri veya hibrit mimariler kullanılarak sınıflandırma başarısının daha da artırılması mümkündür.

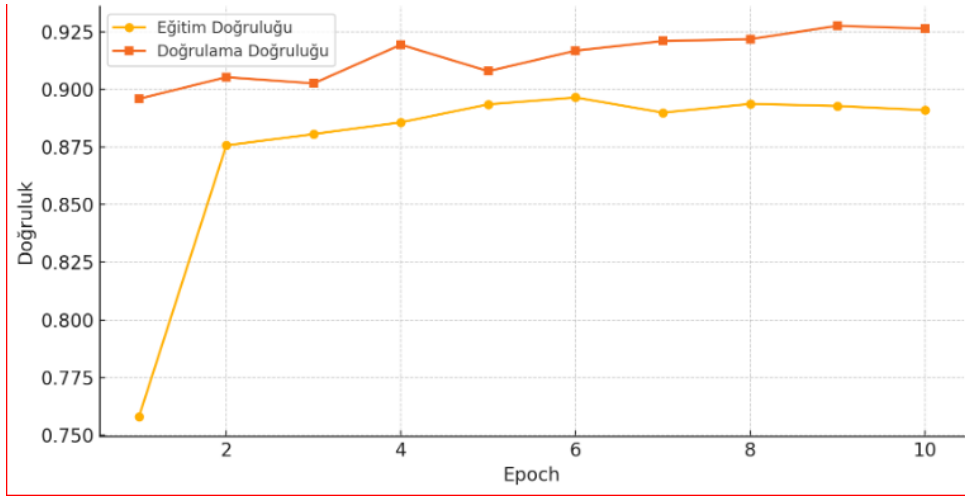
SONUÇ GRAFİKLERİ



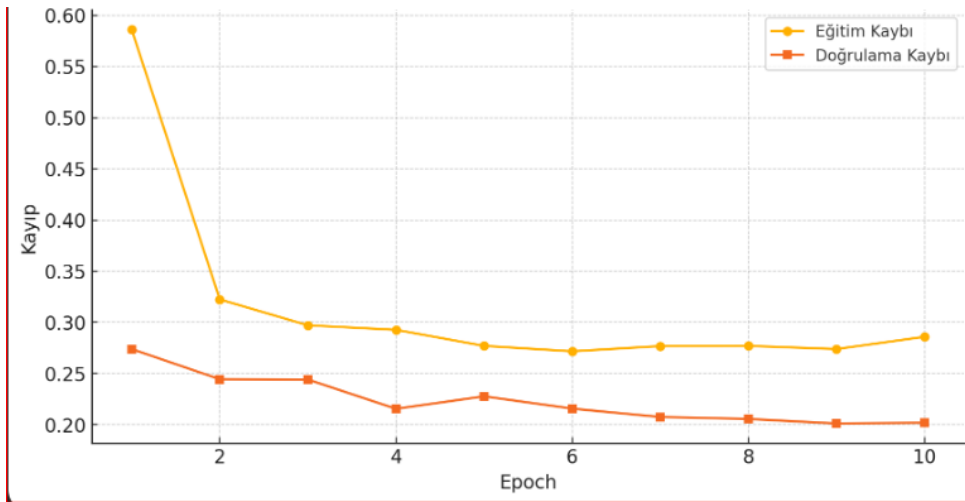
Şekil 3.9.1. ResNet Doğrulama Doğrusu Grafiği



Şekil 3.9.2. ResNet Kayıp Fonksiyonu Grafiği



Şekil 3.10.1. DenseNet Doğrulama Doğrusu Grafiği



Şekil 3.10.2. DenseNet Kayıp Fonksiyonu Grafiği

SONUÇLAR VE ÖNERİLER

Bu çalışmada, Covid-19 tespiti için Covid-19 Radiography Database kullanılarak çeşitli derin öğrenme teknikleri incelenmiş ve en etkili CNN modelleri değerlendirilmiştir. Veri kümesi, Covid-19, normal ve pnömoni olmak üzere üç sınıftan oluşmakta olup, görüntü sınıflandırma işlemi bu sınıflar üzerinden gerçekleştirilmiştir.

Yapılmış olan çalışmalarda gözlemlerim dahilinde veri ön işleme aşamasında, görüntülerin boyutları standartlaştırılmış, normalize edilmiştir ve veri artırma teknikleri uygulanmıştır. Görüntü artırma, veri dengesizliğini gidermek ve modelin genelleme kapasitesini artırmak amacıyla gerçekleştirilmiştir.

Modellerin eğitiminde ve karşılaştırılmasında genel olarak bazı sonuçlara varılmıştır.

DenseNet; yoğun bağlantılı yapısıyla, her katmandaki özelliklerin bir sonraki katmana aktarılmasını sağlamış ve parametre verimliliği ile dikkat çekmiştir. Özellikle karmaşık veri kümelerinde yüksek performans göstermiştir.

ResNet; skip connections (atlamalı bağlantılar) ile derin ağların eğitilmesini kolaylaştırmış ve gradyan kaybolması sorununu büyük ölçüde çözmüştür. Bu özellikleri sayesinde derin mimarilerde dahi istikrarlı bir performans sunmuştur.

LeNet; basit yapısıyla temel bir CNN modeli olan LeNet, giriş düzeyindeki veri sınıflandırma problemlerinde etkili olsa da, daha karmaşık modellerle kıyaslandığında sınırlı bir performans göstermiştir.

AlexNet; daha derin katmanları ve ReLU aktivasyon fonksiyonu sayesinde LeNet'e kıyasla daha yüksek doğruluk sağlamış ancak mobilite açısından sınırlı kalmıştır.

VGGNet; sabit filtre boyutları ve derin katmanları ile yüksek doğruluk sağlamış ancak parametre sayısının fazlalığı nedeniyle eğitim ve uygulama süresi uzamıştır.

InceptionNet; farklı boyutlardaki filtreleri aynı anda kullanabilen Inception modülleri, modelin hem doğruluk hem de verimlilik açısından dikkat çekmesini sağlamıştır.

MobileNet; derin ayrık evrişimler (depthwise separable convolutions) sayesinde hafif bir yapı sunmuş ve düşük güçlü cihazlar için uygun bir çözüm olarak öne çıkmıştır.

EfficientNet; compound scaling (bileşik ölçekleme) yaklaşımı ile doğruluk ve hesaplama maliyeti arasında etkili bir denge kurmuştur.

Bu çalışma, Covid-19, Normal ve Viral Pnömoni sınıflarını akciğer röntgen görüntülerinden otomatik olarak tespit edebilmek amacıyla iki farklı derin öğrenme mimarisi olan ResNet101 ve DenseNet201'in karşılaştırmalı performans analizini sunmuştur. Her iki model, transfer öğrenme yaklaşımıyla ImageNet veri kümesinden önceden öğrenilmiş ağırlıkları kullanarak eğitilmiş; görüntüler, 224x224 boyutunda normalize edilmiş RGB formatında modele aktarılmıştır. Bu bölümde, çalışmanın öğrenme sonuçları özetlenmekte ve gelecek çalışmalara yönelik öneriler detaylandırılmaktadır.

Model eğitimi sonucunda elde edilen metrikler, her iki mimarinin Covid-19 tespitinde kullanılabilir düzeyde başarılı olduğunu göstermiştir. ResNet101, 10 epoch sonunda eğitim verisi üzerinde yaklaşık olarak %89.1 doğruluk oranına ulaşmış, doğrulama doğruluğu ise %92 seviyesinde stabil kalmıştır. Buna karşılık DenseNet201, daha az dalgalanma gösteren ve daha stabil bir doğrulama eğrisi sunarak %93 seviyesinde daha iyi bir doğruluk elde etmiştir. Eğitim kaybı değerleri her iki modelde de eğitim boyunca istikrarlı bir şekilde azalmış, DenseNet201 modelinde bu azalma daha düzenli olmuştur.

Elde edilen classification report ve confusion matrix analizleri, sınıf bazlı başarıların ayrıntılı göstergesini sunmuştur. Hem ResNet101 hem de DenseNet201 modelleri, Covid-19 sınıfını başarılı bir şekilde sınıflandırmıştır. Precision ve recall oranları her iki modelde de bu sınıf için %94'ler seviyesinde seyretmiştir. Ancak, viral pnömoni sınıfında ResNet modelinin daha fazla hata yaptığı, bu sınıfı zaman zaman Covid-19 ile karıştırdığı gözlemlenmiştir. DenseNet modelinde ise bu hata oranı daha düşüktür. F1-score değerleri kıyaslandığında DenseNet %92'nin üzerine çıkarken, ResNet101 modeli %85 civarında kalmıştır.

ResNet101 modeli, derin yapısı ve artık bağlantı mekanizması sayesinde karmaşık verileri öğrenme konusunda başarılıdır. Ancak, eğitim sürecinde doğrulama metriklerinde zaman zaman görülen dalgalanmalardan dolayı modelin belirli durumlarda kararsız kalabildiği ortaya konmuştur. Bu durum, genellikle Viral

Pnömoni gibi sınıflar arasında yapısal benzerliğin fazla olduğu durumlarda gözlemlenmiştir.

DenseNet201 ise her katmanın çıktısını bir sonraki tüm katmanlara aktararak daha verimli bir bilgi akışı sağlamış, hem kayıp fonksiyonu hem de doğruluk eğrileri açısından daha düzenli bir öğrenme süreci göstermiştir. DenseNet modeli aynı zamanda daha az epoch sayısında eğitim süresi açısından da avantajlı duruma geçmiştir.

Her iki modelin pratikteki uygulanabilirliği, tıbbi karar destek sistemleri için oldukça umut vericidir. Özellikle Covid-19 sınıfının doğru tespit edilmesi, birçok sıkıntılı durumda erken izolasyon ve tedavi süreçlerinin başarısı için kritik bir rol oynamaktadır. DenseNet modelinin görece daha az hata ile çalışması, bu modelin sahada daha çok tercih edilebilir olduğuna gösterebilir. Ancak ResNet de Covid-19 sınıfında yeterince başarılı olmuş, sadece viral pnömoni gibi sınıflarda daha dikkatli bir ön işleme veya veri düzenlemesi gereksinimi olmuştur.

Bu çalışma, derin öğrenme modellerinin Covid-19 gibi yayılım hızı yüksek hastalıkların tespiti için etkili alternatifler olduğunu ortaya koymuştur. Ancak model başarısının, kullanılan veri setinin kalitesi ve dengesine bağlı olduğunu da göstermektedir. Örneğin, Viral Pnömoni sınıfındaki yanlış sınıflandırmalar, bu sınıfa ait verilerin yetersizliğine veya bu verilerin yeterince ayrıştırıcı özellik içermemesine bağlı olabilir.

KAYNAKÇA

- [1]..... M. E. H. Chowdhury vd., “Can AI Help in Screening Viral and COVID-19 Pneumonia?”, *IEEE Access*, c. 8, ss. 132665-132676, 2020, doi: 10.1109/ACCESS.2020.3010287.
- [2].....“COV-ECGNET: COVID-19 detection using ECG trace images with deep convolutional neural network | Health Information Science and Systems”. Eriřim: 02 Ocak 2025. [Çevrimiçi]. Eriřim adresi: <https://link.springer.com/article/10.1007/s13755-021-00169-1>
- [3]..... A. Jaiswal ve A. Bist, “Analysis of Deep Learning algorithms on COVID-19 Radiography Database”, *Int. J. Adv. Sci. Technol.*, Oca. 2020, Eriřim: 03 Ocak 2025. [Çevrimiçi]. Eriřim adresi: https://www.academia.edu/43284198/Analysis_of_Deep_Learning_algorithms_on_COVID_19_Radiography_Database
- [4].. A. M. Ismael ve A. řengür, “Deep learning approaches for COVID-19 detection based on chest X-ray images”, *Expert Syst. Appl.*, c. 164, s. 114054, řub. 2021, doi: 10.1016/j.eswa.2020.114054.
- [5]..S. Rahman, S. Sarker, M. A. A. Miraj, R. A. Nihal, A. K. M. Nadimul Haque, ve A. A. Noman, “Deep Learning–Driven Automated Detection of COVID-19 from Radiography Images: a Comparative Analysis”, *Cogn. Comput.*, c. 16, sy 4, ss. 1735-1764, Tem. 2024, doi: 10.1007/s12559-020-09779-5.
- [6]..P. S. Kumar, K. Supriya, M. R. K, ve T. S. K. T. Malisetti, “CoVid-19 Detection leveraging Vision Transformers and Explainable AI”, 06 Mayıs 2024, *arXiv*: arXiv:2307.16033. doi: 10.48550/arXiv.2307.16033.
- [7]. “Automated detection of COVID-19 through convolutional neural network using chest x-ray images | PLOS ONE”. Eriřim: 03 Ocak 2025. [Çevrimiçi]. Eriřim adresi: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0262052>

- [8]..... W. Liu *vd.*, “Complex network-based classification of radiographic images for COVID-19 diagnosis”, *PLOS ONE*, c. 18, sy 9, s. e0290968, Eyl. 2023, doi: 10.1371/journal.pone.0290968.
- [9] O. Fedoruk, K. Klimaszewski, A. Ogonowski, ve M. Kruk, “Additional Look into GAN-based Augmentation for Deep Learning COVID-19 Image Classification”, *Mach. Graph. Vis.*, c. 32, sy 3/4, ss. 107-124, Ara. 2023, doi: 10.22630/MGV.2023.32.3.6.
- [10].....A. Hamza *vd.*, “COVID-19 classification using chest X-ray images based on fusion-assisted deep Bayesian optimization and Grad-CAM visualization”, *Front. Public Health*, c. 10, Kas. 2022, doi: 10.3389/fpubh.2022.1046296.
- [11]... Primanshu, G. Chhikara, P. Rana, ve Nitin, “Transfer learning based Covid-19 detection using Radiography Dataset”, içinde *2022 2nd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE)*, Nis. 2022, ss. 2134-2139. doi: 10.1109/ICACITE53722.2022.9823611.
- [12]“COVID-19 Detection from Chest X-ray Images Using Feature Fusion and Deep Learning”. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://www.mdpi.com/1424-8220/21/4/1480>
- [13].....S. Minaee, R. Kafieh, M. Sonka, S. Yazdani, ve G. Jamalipour Soufi, “Deep-COVID: Predicting COVID-19 from chest X-ray images using deep transfer learning”, *Med. Image Anal.*, c. 65, s. 101794, Eki. 2020, doi: 10.1016/j.media.2020.101794.
- [14].... M. A. Salam, S. Taha, ve M. Ramadan, “COVID-19 detection using federated machine learning”, *PLOS ONE*, c. 16, sy 6, s. e0252573, Haz. 2021, doi: 10.1371/journal.pone.0252573.
- [15].. Shivam, “Introduction to DenseNets (Dense CNN)”, Analytics Vidhya. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://www.analyticsvidhya.com/blog/2022/03/introduction-to-densenets-dense-cnn/>
- [16] G. Huang, Z. Liu, L. van der Maaten, ve K. Q. Weinberger, “Densely Connected Convolutional Networks”, program adı: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, ss. 4700-4708. Erişim: 03 Ocak

2025. [Çevrimiçi]. Erişim adresi:

https://openaccess.thecvf.com/content_cvpr_2017/html/Huang_Densely_Connected_Convolutional_CVPR_2017_paper.html

[17].....K. He, X. Zhang, S. Ren, ve J. Sun, “Deep Residual Learning for Image Recognition”, 10 Aralık 2015, *arXiv*: arXiv:1512.03385. doi: 10.48550/arXiv.1512.03385.

[18].....T. SHARMA, “Detailed Explanation of Residual Network(Resnet50) CNN Model.”, Medium. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://medium.com/@sharma.tanish096/detailed-explanation-of-residual-network-resnet50-cnn-model-106e0ab9fa9e>

[19].....“Papers with Code - Bottleneck Residual Block Explained”. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://paperswithcode.com/method/bottleneck-residual-block>

[20].....S. Bangar, “Resnet Architecture Explained”, Medium. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://medium.com/@siddheshb008/resnet-architecture-explained-47309ea9283d>

[21]..K. Simonyan ve A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition”, 10 Nisan 2015, *arXiv*: arXiv:1409.1556. doi: 10.48550/arXiv.1409.1556.

[22] S. Bangar, “VGG-Net Architecture Explained”, Medium. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://medium.com/@siddheshb008/vgg-net-architecture-explained-71179310050f>

[23].... “Fig. 6 Architecture of pre-trained VGG-16 (Left) and VGG-19 (Right) [61]”, ResearchGate. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: https://www.researchgate.net/figure/Architecture-of-pre-trained-VGG-16-Left-and-VGG-19-Right-61_fig3_355177485

[24].....A. Krizhevsky, I. Sutskever, ve G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks”, içinde *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2012. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi:

https://papers.nips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html

[25].. S. Bangar, “AlexNet Architecture Explained”, Medium. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://medium.com/@siddheshb008/alexnet-architecture-explained-b6240c528bd5>

[26] Manasabrao, “AlexNet Architecture Detailed Explanation”, Medium. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://medium.com/@manasabrao2020/alexnet-architecture-detailed-explanation-52427cc22ce5>

[27].....C. Szegedy *vd.*, “Going Deeper with Convolutions”, 17 Eylül 2014, *arXiv*: arXiv:1409.4842. doi: 10.48550/arXiv.1409.4842.

[28].....N. Malingan, “Introduction to InceptionNet”, Scaler Topics. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://www.scaler.com/topics/inception-network/>

[29]...S. Bangar, “LeNet 5 Architecture Explained”, Medium. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://medium.com/@siddheshb008/lenet-5-architecture-explained-3b559cb2d52b>

[30]...M. Gazar, “LeNet-5 in 9 lines of code using Keras”, Medium. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://medium.com/@mgazar/lenet-5-in-9-lines-of-code-using-keras-ac99294c8086>

[31]. N. Malingan, “Convolutional Neural Networks - LeNet”, Scaler Topics. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://www.scaler.com/topics/lenet/>

[32] “Efficientnet Architecture”, GeeksforGeeks. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://www.geeksforgeeks.org/efficientnet-architecture/>

[33] M. Tan ve Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks”, 11 Eylül 2020, *arXiv*: arXiv:1905.11946. doi: 10.48550/arXiv.1905.11946.

[34].....A. K. Ph.D, “Ölçeklendirme ile CNN Modelinin Doğruluk ve Verimliliğini Artırma: EfficientNet”, Medium. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://ayyucekizrak.medium.com/%C3%B6l%C3%A7eklendirme-ile-cnn-modelinin-do%C4%9Fruluk-ve-verimlili%C4%9Fini-art%C4%B1rma-efficientnet-cb6f2b6512de>

[35]....A. G. Howard *vd.*, “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications”, 17 Nisan 2017, *arXiv*: arXiv:1704.04861. doi: 10.48550/arXiv.1704.04861.

[36].....S. PA, “An Overview on MobileNet: An Efficient Mobile Vision CNN”, Medium. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://medium.com/@godeep48/an-overview-on-mobilenet-an-efficient-mobile-vision-cnn-f301141db94d>

[37]N. AI, “MobileNet”, Medium. Erişim: 03 Ocak 2025. [Çevrimiçi]. Erişim adresi: <https://medium.com/@nocodingai/mobilenet-fc34af9f58a5>

ÖZGEÇMİŞ

Hatice Akıllı, Aralık-2000 tarihinde Bursa’da doğmuştur. İlkokul, ortaokul ve lise eğitimini Bursa’da tamamlamış olup şu an Kocaeli Üniversitesi Bilgisayar Mühendisliği son sınıf öğrencisidir. Bitirme çalışması olarak ilgi alanı olan yapay zeka alanında bir proje yapmak istemiştir.

